LogShield: Detection and Tracking of Secret Exposure in Kubernetes Logging Systems

Submitted in partial fulfilment of the requirement for the award of Degree of Bachelor of Technology in Computer Science

Submitted To



SVKM's NMIMS,
Mukesh Patel School of Technology Management & Engineering,
Shirpur Campus (M.H.)

Submitted By:
Prince Roy (Enrollment No. 70552300031)
Arnab Chowdhury (Enrollment No. 70552300032)
Rounak Neema (Enrollment No. 70552300046)
Manas Borse (Enrollment No. 70552300059)

Under The Supervision Of:
Prof. Rudrani Ingle

DEPARTMENT OF COMPUTER SCIENCE
Mukesh Patel School of Technology Management & Engineering
ACADEMIC SESSION: 2026-27

CERTIFICATE



This is to certify that the capstone project entitled "LogShield: Detection and Tracking of Secret Exposure in Kubernetes Logging Systems" has been done by

Prince Roy (Enrollment No. 70552300031)

Arnab Chowdhury (EnrollmentNo.70552300032)

Rounak Neema (Enrollment No. 70552300046)

Manas Borse (Enrollment No. 70552300059)

under my guidance and supervision & has been submitted partial fulfillment of the degree of "**Bachelor of Technology** in **Computer Science"** of SVKM'S NMIMS (Deemed-to-be university), MPSTME Shirpur Campus (M.H.), India.

Prof. Rudrani Ingle	**Examiner**
Project Mentor	
(Internal Guide)	
	Dr. Nitin Choubey
Date:	**Head of Department**
Place: Shirpur	

DEPARTMENT OF COMPUTER SCIENCE

Mukesh Patel School of Technology Management & Engineering

ACKNOWLEDGEMENT

After the completion of this capstone Project work, words are not enough to express my feelings about all those who helped me to reach my goal; feeling above this is my indebtedness to The Almighty for providing me this moment in life.

It's a great pleasure and moment of immense satisfaction for me to express my profound gratitude to **Prof. Rudrani Ingle**, Computer Science Department, MPSTME, Shirpur, whose constant encouragement enabled me to work enthusiastically. Their perpetual motivation, patience and excellent expertise in discussion during progress of the project work have benefited me to an extent, which is beyond expression. Their depth and breadth of knowledge of Computer Science field made me realize that theoretical knowledge always helps to develop efficient operational software, which is a blend of all core subjects of the field. I am highly indebted to them for their invaluable guidance and ever-ready support in the successful completion of this project in time. Working under their guidance has been a fruitful and unforgettable experience.

We express my sincere thanks and gratitude to **Dr. Nitin Choubey**, Head of Department, Computer Science, MPSTME, Shirpur, for providing necessary infrastructure and help to complete the project work successfully.

We also extend my deepest gratitude to **Dr. Venkatadri M**, Associate Dean, SVKM'S NMIMS, Shirpur Campus for providing all the necessary facilities and true encouraging environment to bring out the best of my endeavors.

We sincerely wish to express my grateful thanks to all members of the staff of computer science department and all those who have embedded me with technical knowledge of computer technology during various stages of B.Tech. Computer Science.

We would like to acknowledge all my friends, who have contributed directly or indirectly in this capstone Project work.

The successful completion of a capstone Project is generally not an individual effort. It is an outcome of the cumulative effort of a number of persons, each having their own importance to the objective. This section is a vote of thanks and gratitude towards all those persons who have directly or indirectly contributed in their own special way towards the completion of this project.

Prince Roy
Arnab Chowdhury
Rounak Neema
Manas Sandeep Borse

ABSTRACT

Modern cloud applications frequently leak sensitive credentials into telemetry streams, exposing database passwords, API access keys, and authentication tokens during routine operation. Existing defenses fail to resolve this channel effectively: static scanners only inspect source code before deployment, while centralized log services analyze records only after they leave the server, leaving plaintext secrets exposed across networks, third-party dashboards, and persistent storage. This project presents LogShield, an automated security system deployed directly alongside application containers to intercept and sanitize log streams at runtime. LogShield inspects outgoing lines in real time, redacts exposed credentials, and stores the original plaintext in an encrypted local vault backed by a tamper-evident audit ledger for authorized retrieval. When an active leak is detected, an autonomous containment engine immediately quarantines the leaking container at the network level, halting further exposure without disrupting neighboring microservices. In benchmarks across 9,205 production log samples, LogShield achieved 99.9% detection accuracy with zero false alarms, maintained an average latency of 0.50 milliseconds per line, and completed network isolation within 30 milliseconds. By preventing credential leakage at the source without requiring application code modifications, LogShield provides a practical, compliance-ready defense for modern containerized environments.

Keywords: Cloud security, credential leakage, runtime log sanitization, container isolation, data privacy.

TABLE OF CONTENT

Sr. No.	Chapter / Section	Page
	Certificate	ii
	Acknowledgement	iii
	Abstract	iv
	Table of Contents	v
	List of Figures	viii
	List of Tables	x
1	**CHAPTER 1: INTRODUCTION**	**1**
1.1	Background	1
1.2	Motivation	2
1.3	Objectives	3
1.4	Scope of the Project	4
1.5	Overview of the Proposed System	5
1.6	Technology Stack	6
1.7	Organisation of the Report	7
2	**CHAPTER 2: LITERATURE SURVEY**	**9**
2.1	Secret Leakage in Software Systems and Logs	9
2.2	Secret Detection Approaches	9
2.3	Existing Approaches to Protecting Secrets	11
2.4	SecretBench and Pattern Corpora	13
2.5	Multi-Pattern Matching: The Aho-Corasick Automaton	13
2.6	Confidence Scoring and Weight Learning	14
2.7	Encrypted Storage, Keyed Hashing and Secrets Management	14
2.8	Kubernetes Sidecars and Admission Control	14
2.9	Preliminary Research by the Project Team	15
2.10	Research Gap and Contribution	19
3	**CHAPTER 3: PROBLEM DEFINITION AND PROPOSED SOLUTION**	**22**
3.1	Problem Statement	22
3.2	Needs Analysis	22
3.3	Threat Model	25

Sr. No.	Chapter / Section	Page
3.4	Proposed Solution	26
3.5	Feasibility Study	27
3.6	Methodology	27
3.7	Key Design Decisions	29
4	**CHAPTER 4: SYSTEM DESIGN AND MATHEMATICAL FORMULATION**	**29**
4.1	System Architecture	29
4.2	The Six-Stage Detection Pipeline	30
4.3	Mathematical Formulation and Complexity Breakdown	32
4.4	Core Data Structures	34
4.5	Masking and Vaulting Sequence	35
4.6	SecureReveal Vault and Forward-Secure Audit Ledger	36
4.7	SecretLineage and Scoped Exemption Engine	37
4.8	Autonomous Closed-Loop Containment Engine	38
4.9	Rollout and Webhook Architecture	38
4.10	Error Handling and Fail-Open Policy	40
5	**CHAPTER 5: RESULT ANALYSIS AND EMPIRICAL EVALUATION**	**41**
5.1	Experimental Environment	41
5.2	Authoritative Dataset and Experiment Inventory	41
5.3	Machine Learning Model Training and Statistical Grounding	42
5.4	Controlled 6-Stage Ablation Analysis	43
5.5	Adversarial Evasion Benchmark on 2,400 Labeled Samples	44
5.6	Live Kubernetes Cluster Containment and Phase-Defined Latencies	42
5.7	Multi-Baseline Comparative Evaluation	47
5.8	Operational Latency and Throughput Regimes	47
5.9	Hard-Negative Suppression	48
5.10	Snapshots of the System	49
5.11	Implementation Completion Status	49
6	**CHAPTER 6: TESTING AND SECURITY VERIFICATION**	**51**
6.1	Testing Strategy	51

Sr. No.	Chapter / Section	Page
6.2	Unit Tests	51
6.3	Engine Edge-Case Verification (EC-01 to EC-15)	52
6.4	Adversarial Robustness and Security Edge Cases	55
6.5	Live Five-Microservice Cluster Testbed	55
6.6	Shortcomings of Testing and Threats to Validity	56
7	**CHAPTER 7: CONCLUSION AND FUTURE WORK**	**57**
7.1	Conclusion	57
7.2	Summary of Technical Contributions	57
7.3	Architectural Advantages, Shortcomings and Applications	58
7.4	Future Research Directions	59
	References	**61**
	Appendix A: Production Sample Code	**63**
	Appendix B: Data Sheets and Database Schemas	**65**
	Appendix C: List of Constituents and Specifications	**69**
	Appendix D: Research Papers Presented and Published	**72**

LIST OF FIGURES

Figure No.	Title	Page
1.1	Before and after: secret redaction in action	2
1.2	Points of action for existing tooling versus LogShield in-pod interception	3
2.1	Eight-pillar secret-detection capability taxonomy and LogShield implementation coverage	20
3.1	Threat model: opponents A1 - A5 and the defenses LogShield enforces in their path	26
4.1	LogShield sidecar architecture within a Kubernetes pod boundary	30
4.2	The six-stage sequential detection pipeline with O(N) early-exit filtering	31
4.3	Three-pass layered arrangement of Stage 4 Secret Classifier	32
4.4	Tri-state operational decision gate boundaries (PASS, FLAG, REDACT)	32
4.5	Logical data model SCSResult, VaultEntry, FingerprintRecord and AuditEvent	35
4.6	Sequence flow from container stdout write to vault storage and sanitized emission	36
4.7	SecretLineage bipartite graph: cross-pod sprawl mapping using canonical blind HMAC fingerprints	37
4.8	Closed-loop autonomous containment lifecycle: real-time blast radius correlation and V1NetworkPolicy egress quarantine	38
4.9	Kubernetes mutating admission webhook automated injection workflow	39
5.1	Machine learning model training convergence (L-BFGS), ROC/PR curves, and zero-FP threshold calibration	43
5.2	Controlled ablation study: Precision, Recall and F1-score across the pipeline stages (N=9,205)	44
5.3	Adversarial evasion benchmark, including Baseline Scanner vs. LogShield recall, across the five obfuscation vectors (N=2,400)	46
5.4	Processing latency breakdown across the pipeline stages and throughput scaling across the payload sizes	48
5.5	Sidecar console output, including live credential interception and [REDACTED:SEC-XXXX:SCS=99] masking	49

Figure No.	Title	Page
5.6	SecureReveal CLI: retrieve, sprawl-report, and forward-secure audit log verification	49
5.7	Multi-container pod in Minikube: application and LogShield sidecar running in 2/2 Ready state	49
6.1	Verification status breakdown across the fifteen engine edge cases (EC-01 to EC-15)	55
7.1	Summary of five major technical advancements of the LogShield capstone project	58

LIST OF TABLES

Table No.	Title	Page
1.1	Scope boundaries of the project	5
1.2	Secret Confidence Score decision gates	6
1.3	Technology stack applied in LogShield	6
2.1	Review of current work in secret detection	10
2.2	Existing solution categories and their shortcoming relative to runtime log leakage	12
2.3	Comparison of secret-protection approaches	12
2.4	Review of relevant research papers and their contributions	16
2.5	Eight-pillar taxonomy of secret-detection capability and LogShield coverage	20
3.1	Functional requirements	22
3.2	Non-functional needs	24
3.3	Development and deployment phases	27
3.4	Key design decisions and justification	29
4.1	Feature space and parameter weights for the 6-factor SCS model	33
4.2	Principal data structures	34
4.3	Vault and lineage component characteristics	36
4.4	Kubernetes resource roles	39
5.1	Environment employed for measurements and demonstration	41
5.2	Authoritative inventory of evaluation corpora	42
5.3	Controlled 6-stage ablation study	39
5.4	Adversarial evasion benchmark breakdown across the obfuscation categories	45
5.5	Broad architectural and quantitative comparison, including LogShield vs. industry baselines	47
5.6	Reconciled latency and throughput across the operational regimes	47
5.7	Cryptographic and lineage latency profile	48
5.8	Component completion status at final review	45
6.1	Levels of testing and automated verification	51
6.2	Engine edge cases EC-01 to EC-15 and verification status	52

Table No.	Title	Page
7.1	Industry applications of LogShield	59
B.1	Detection engine fixed size parameters	65
B.2	Vault entry schema (SQLite table: vault)	66
B.3	Fingerprint record schema (SQLite table, namely fingerprints)	66
B.4	Audit ledger schema (SQLite table: audit_ledger)	67
C.1	Software components, versions and licenses	69

CHAPTER 1: INTRODUCTION

1. BACKGROUND

i. The Problem of Secrets in Logs

Millions of log lines daily are created by distributed cloud-native microservices to support observability, application performance monitoring, and compliance audits, at least for now. In production environments, engineering teams routinely are logging unredacted credentials during the service startup, database connection pooling, third-party API integration, and unhandled exception handling, in the main. Representative examples include database URIs (RFC 3986 [47]) containing embedded passwords logged at debug levels, full HTTP request headers carrying bearer tokens, third-party payment gateway private keys, and environment variable dumps yielded during the process initialization; this security weakness is formally cataloged as CWE-532 (Insertion of Sensitive Information into Log File) [23] (and studied extensively across enterprise telemetry by Barik et al. [42]). Industry regulatory standards, including PCI-DSS Requirement 3.3 [25], HIPAA Security Rule 45 CFR Part 164 [52], European Union GDPR [40], NIS 2 Directive [35], and the OWASP Logging Cheat Sheet [22], unambiguously command that sensitive authentication tokens and cryptographic keys must never appear in persistent log records.

ii. Where a Leaked Secret Goes

The record to the local host filesystem is written by the host container runtime (such as containerd or CRI-O) [9], [41], when any application writes one unredacted credential to its standard output stream (stdout), in many respects. In light of this dynamic, all all the container log files are tailed by node-level log daemons, such as Fluent Bit [21] or Fluentd, and ship the unencrypted records across the cluster network to centralized aggregation tiers, including Elasticsearch, Grafana Loki, Amazon CloudWatch [17], Splunk, or Datadog [7], on the whole; once stored in centralized storage, the leaked secret is duplicated across the cold backups, indexed in search clusters, and mirrored onto developer monitoring dashboards. The credential remains visible to every employee or third-party contractor broadly permitting; in turn, under the classic operational workflows, weeks often are passing before the organization spots

one more second credential leak; and thus, by that time, remediation needs rotating the exposed key across all dependent services, invalidating active client sessions, and conducting digital forensics to verify whether threat actors exploited this key.

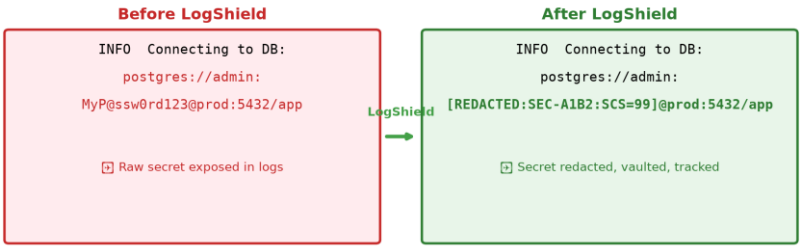


Figure 1.1 :Before and after: secret redaction in action

iii. The Gap in Existing Tooling

Thorough research is revealing that secret leakage remains widespread in version control systems and cloud infrastructure, on the whole; prior studies, such as the work by Meli et al. [38], the SecretBench dataset [27], and GitGuardian sprawl telemetry [11], stronged mostly on static analysis of git commit histories and source code repositories, on the whole. Yet any completely separate attack surface is embodied by runtime container log streams, so existing tooling leaves one major structural gap, namely static scanners are inspecting code too early (pre-deployment); simultaneously, centralized cloud log sanitizers are inspecting entries too late (post-egress). Under these conditions, no open-source, container-native runtime engine existed to intercept and sanitize standard output streams inside the pod before network transmission while preserving recovery capabilities for authorized incident response, and log lines are gathered dynamically from runtime variables, environment configuration, and third-party API responses that never exist in static source files.

2. MOTIVATION

The motivation for LogShield originates from the realization that current security tooling operates either too far from the point of leakage or too late in the attack lifecycle, in many respects; static repository scanners such as Gitleaks [13], TruffleHog [12], and detect-secrets [14] analyse git histories after developer commits arise, on the whole. Along these dimensions, secrets managers such as HashiCorp Vault [15] and AWS

Secrets Manager [16] are providing secure storage for credentials at rest but can't prevent an application from accidentally printing an acquired key to stdout. Runtime security monitors like Falco [20] inspect Linux system calls to detect behavioral oddities but do not read application telemetry; centralized cloud data sanitizers, such as Datadog Sensitive Data Scanner [8] or AWS CloudWatch data protection policies [17], process logs only after network transmission to the provider's ingestion tier.

A second motivation is the urgent need to transition runtime logging defence from passive string masking to active, closed-loop mitigation, in real-world scenarios; existing pipeline redactors simply overwrite matching strings without offsetting the compromised credential. LogShield was conceived to join pre-ingestion interception, multi-factor confidence scoring, recoverable encrypted vaulting, transformation-aware cross-service lineage tracking, and autonomous Kubernetes control-plane containment into a person lightweight sidecar container rolled out directly inside the pod, all things considered.

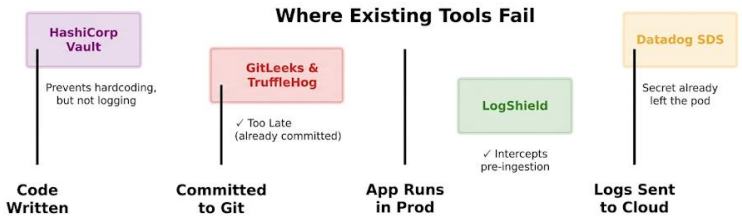


Figure 1.2 Points of action for existing tooling versus LogShield in-pod interception

3. OBJECTIVES

To tackle the shortcomings of prior frameworks, we specified eight concrete, empirically verifiable engineering objectives for the LogShield project:

- In-Pod Zero-Copy Interception: Intercept and sanitize application stdout streams inside the pod boundary applying a shared memory-backed volume, preventing raw credentials from crossing cluster network interfaces, at least in principle.
- Multi-Signal Confidence Scoring: Execute a calibrated Secret Confidence Score (SCS) from 0 to 100 combining Shannon entropy [58], variable-name proximity, structural checksums, contextual trigger words, and encoding awareness, in actual application.
- Adaptive Evasion Resistance, namely Design a sliding-window Token Fusion preprocessor to defeat adversarial delimiter-split and hex-escaped obfuscation patterns that bypass standard regex engines, in real-world scenarios.

- Under 1.0 ms, Sub-Millisecond Processing Latency, namely Preserve an average per-line processing latency while sustaining single-core throughput exceeding 4,500 lines per second, all things considered.
- Ephemeral Encrypted Recovery, namely Store masked credentials in a local, AES-128-CBC [50] and HMAC-SHA256 [43], [51] verified vault, retrievable only by authorized operators employing unique reference markers paired with a tamper-evident audit ledger, in real-world scenarios.
- Privacy-Preserving Sprawl Tracking, such as Track cross-service credential increase applying canonical blind HMAC-SHA256 fingerprint [43], [51]s [43], [51], across raw, Base64, and URL encodings without keeping secondary plaintext copies, broadly speaking.
- Autonomous Closed-Loop Containment, including Integrate an automated control-plane response engine that blends dynamic Kubernetes V1NetworkPolicy egress rules [10] to isolate compromised pods, within milliseconds of leak detection, in the main.
- Zero-Configuration Developer Experience, namely Allow seamless cluster-wide rollout, through a Kubernetes mutating admission webhook, injecting sidecar containers automatically grounded in namespace labels without needing code changes, in the main.

4. SCOPE OF THE PROJECT

i) In Scope

The completed deployment encompasses the Python 3.11 sidecar daemon, the six-stage LogMask detection pipeline, the Aho-Corasick keyword prefilter [57], the Token Fusion evasion normalizer, the 1,361-pattern layered classifier, the L2-regularized logistic regression confidence engine, the SecureReveal encrypted vault, the forward-secure SHA-256 audit ledger, the SecretLineage blind fingerprint tracker, this Scoped Exemption Engine, the Autonomous Containment Engine with Kubernetes V1NetworkPolicy integration, the mutating admission webhook manifests, and an end-to-end evaluation harness spanning four benchmark datasets and an isolated five-microservice cluster, in actual application.

ii) Out of Scope

The project deliberately excludes in-kernel eBPF socket-layer interception, multi-cluster distributed consensus protocols for cross-cloud lineage, broad

enterprise PII policy engines, and hardware security module (HSM) enclave integrations; those architectural extensions are frequently spotted as future research directions in Chapter 7, in real-world scenarios; table 1.1 summarises the boundary.

Table 1.1: *Scope boundaries of the project*

Within the scope of this project	Out of scope (future research)
In-pod sidecar detection engine (six-stage pipeline)	Centralized cross-cluster distributed consensus
1,361+ regex patterns (600 catalog + 761 SecretBench)	eBPF kernel sockops hook for sidecar-less interception
Adaptive Evasion Defense (sliding-window Token Fusion)	Hardware security module (HSM) enclave binding (SGX/SEV)
Local Fernet encrypted vault with forward-secure hash chaining	Broad enterprise PII policy engines (GDPR address/SSN)
Machine-learned weights fit via L2 regularized logistic regression	Full SIEM analytical engine (only audit exporter is built)
Autonomous closed-loop containment via Kubernetes V1NetworkPolicy	Post-quantum lattice-based blind fingerprinting
Mutating admission webhook for transparent automatic injection	Production-grade multi-cluster certificate authority automation

5. OVERVIEW OF THE PROPOSED SYSTEM

LogShield rolls out as a transparent sidecar container alongside the primary application container [9], [41], within a Kubernetes pod, in real-world scenarios. A local memory-backed emptyDir storage volume is shared by both containers. The application writes its telemetry to this volume, where LogShield tails the stream in real time, at least for now. A tri-state policy gate is being followed by operational decisions, laid out in Table 1.2. Viewed through this lens, each individual line is traversing a six-stage pipeline that is computing an integer Secret Confidence Score (SCS) from 0 to 100.

Table 1.2: *Secret Confidence Score decision gates*

SCS Range	Operational Decision	Enforced System Action
80 and above	REDACT	Redact token in-place, encrypt in local vault, record blind HMAC lineage, evaluate autonomous containment
50 to 79	FLAG	Tag record with [FLAGGED:SEC-XXXX:SCS=S] for SIEM alerting while preserving plaintext for operational debugging
Below 50	PASS	Allow log line to cross stdout untouched as a corroborated harmless record

6. TECHNOLOGY STACK

Table 1.3: *Technology stack used in LogShield*

System Concern	Selected Technology	Function in the System Architecture
Core Language	Python 3.11.9	Sidecar daemon, detection pipeline, vault, lineage and containment engine
Keyword Prefilter	pyahocorasick (Aho-Corasick)	Stage 2 early-exit search over 2,000 sensitive root keywords
Pattern Matching	Python re (standard library)	Stage 4 three-pass evaluation of 1,361 gathered regular expressions
Confidence Model	scikit-learn, scipy	L2 regularized logistic regression fitting and probability calibration
Authenticated Encryption	cryptography (Fernet)	AES-128-CBC encryption and HMAC-SHA256 authentication for vault records

System Concern	Selected Technology	Function in the System Architecture
Local Data Storage	SQLite (WAL mode)	Embedded storage for vault ciphertexts, lineage graphs, and audit ledgers
Keyed Lineage Fingerprint	hmac (HMAC-SHA256)	Canonical blind keyed fingerprinting for privacy-preserving sprawl tracking
Control-Plane Orchestration	kubernetes Python client	Dynamic V1NetworkPolicy synthesis and pod quarantine enforcement
Microservice Simulation	FastAPI, uvicorn	Multi-microservice testbed creating realistic transaction and leak streams
Local Cluster Platform	Minikube v1.33 / Calico CNI	Isolated Kubernetes control-plane testbed for closed-loop containment
Metrics Telemetry	prometheus_client	Prometheus /metrics endpoint exposing latency histograms and fail-open counters
Test Automation	Pytest	42-test automated test suite covering unit, security and edge-case validation

7. ORGANISATION OF THE REPORT

That capstone report is organized into the seven core chapters, and chapter 3 formalizes the challenge definition, specifies functional and non-functional needs, and shows the threat model, at least empirically. Chapter 2 reviews related literature across the secret detection, regex speedup, and Kubernetes container architectures, on the whole; indeed, also chapter 4 introduces the complete system architecture, mathematical formulations, algorithmic flows, and data schemas. Chapter 5 is supplying broad empirical results across our four evaluation corpora, ML training validation, ablation analysis, evasion benchmarks, and live cluster containment; indeed, chapter 6 details our 42-test validation suite, engine edge-case verification (EC-01 to EC-15), and adversarial

security testing. Viewed through this lens, chapter 7 is concluding the report with a discussion of architectural strengths, production applications, and future research avenues; references and technical appendices are following.

CHAPTER 2: LITERATURE SURVEY

1. SECRET LEAKAGE IN SOFTWARE SYSTEMS AND LOGS

Empirical research by Meli et al. [38] and annual telemetry from GitGuardian [11] corroborate that software credentials leak persistently across modern engineering lifecycles. Meli et al. [38] assessed billions of commits across public GitHub repositories, consistent with configuration leak patterns documented by Niu et al. [32], spotting hundreds of thousands of active API keys, database credentials, and cryptographic certificates, in many respects. Their findings proved that secret leakage is, for all practical purposes, not an occasional aberration but a pervasive result of distributed continuous integration workflows; this vulnerability is exponentially expanded by container logging in microservice environments. Within this framework, unlike static source code repositories where commit histories can, in practice, be rewritten or squashed, log aggregation pipelines forward entries, across the networks within the seconds, broadcasting compromised credentials across the multiple centralized indexes. Developers depend on verbose logging during the troubleshooting, routinely packing runtime context objects, HTTP request headers, and configuration structures directly to standard output.

2. SECRET DETECTION APPROACHES

i. Pattern-Based Detection

Pattern matching applying regular expressions is the primary detection mechanism applied by industry tools, spanning Gitleaks [13] and TruffleHog [12] (and detect-secrets [14]), in actual application, and when cloud service providers issue API keys with distinctive prefix signatures (e.g., AKIA for AWS IAM keys or sk_live_ for Stripe secret keys), regular expressions are reaching near-perfect precision, at least empirically. Viewed through this lens, still pattern-based detection crashes on generic credentials that lack vendor-specific prefixes, and also, severe Regular Expression Denial of Service (ReDoS) vulnerabilities, are introduced by sophisticated regular expressions with nested quantifiers causing rapid backtracking when judging malformed input lines.

ii. Entropy-Based Detection

To spot unstructured credentials, Several tools assess the Shannon entropy [58] of candidate strings; in turn, still, Shannon entropy alone is insufficient, as

empirical studies by Huang et al. [6] demonstrate that secret tokens follow a Gaussian distribution requiring contextual normalization is a flawed stand-in for confidentiality, at least for now; because, cryptographic keys are created from pseudo-random number generators, high information density is showed by them (strategying 4.0 to 5.0 bits per character), broadly speaking. In parallel with this, rolling out entropy-only filters in production log streams creates overwhelming false-positive rates, directing to severe alert fatigue, and harmless non-secret identifiers, incorporating UUIDv4 session identifiers, Git commit hashes, MD5 content checksums (ETags), and Base64-encoded configuration blobs, reveal equivalent entropy profiles.

iii. Setting and Machine-Learning Strategies

The critical need for contextual check was established by SecretBench [27], a benchmark corpus including 97,479 candidate secrets pulled across 818 stores, in actual application. Manual verification disclosed that only 15,084 candidates (15.5%) were authentic secrets, meaning that more than five out of six alerts emitted by standard detectors were false positives, which exposes that isolated pattern matches or entropy scores can't reliably differentiate between gentle identifiers and production secrets, at least in practice. Detectors must assess proximity to sensitive variable names and gauge syntactic arrangement before triggering redaction.

iv. Runtime Log Sanitisation

Log redaction features, such as Datadog Sensitive Data Scanner [8] and AWS CloudWatch data protection policies [17] are offered by commercial logging platforms (alongside Amazon Macie [18] and Google Cloud Sensitive Data Protection [19]), at least in practice. Still managed cloud scanners operate after telemetry exits the customer's Kubernetes cluster, creating an unavoidable window of transit exposure, in real-world scenarios. Against this backdrop, in-pipeline processors are running within the customer's VPC but operate in a centralized aggregation tier, leaving pod-to-node communication unprotected and lacking encrypted recovery or closed-loop containment.

Table 2.1: *Review of current work in secret detection*

Category	Key Studies and Tools	Detection Approach	Critical Limitation
Static Regex Scanners	Gitleaks [13], TruffleHog [12], detect-secrets [14]	Pattern matching against git commit history	Post-commit assessment; can't intercept dynamic runtime log streams
Entropy-Based Detectors	Shannon entropy thresholds [58] in TruffleHog [12]	High information density as proxy for keys	Extreme false-positive rates on gentle UUIDs, ETags and Base64 blobs
Empirical Benchmark Corpora	SecretBench (Basak et al., MSR 2023) [27]	97,479 candidate secrets from 818 repos	Source-code concentrated corpus; does not gauge runtime container telemetry
Managed Cloud Redaction	Datadog Sensitive Data Scanner [8], Macie [18]	Rules evaluated in provider cloud ingestion	Data leaves cluster in plaintext before redaction; no recoverable vault
In-Pipeline Aggregators	Datadog Observability Pipelines Worker [7]	Centralized aggregation tier masking	Separate infrastructure tier; lacks in-pod isolation and closed-loop containment
In-Pod Runtime Sanitizer	LogShield (This Capstone Project)	Pre-ingestion sidecar with autonomous response	Tackles the spotted gap, including in-pod interception with closed-loop mitigation

3. EXISTING APPROACHES TO PROTECTING SECRETS

i. Categories of Tooling

Existing secret defense mechanisms are sorted into the five separate architectural tiers, namely git history scanners [12], [13], [14], secrets managers [15], [16], runtime security platforms [20], cloud log scanners [8], [17], [18], [19], and in-environment log pipelines [7], in many respects. Viewed through this lens, none of these existing categories is joining in-pod stream interception,

ML confidence scoring, recoverable vaulting, and automated pod quarantine, in real-world scenarios.

Table 2.2: *Existing solution categories and their shortcoming relative to runtime log leakage*

Solution Category	Representative Tools	Operational Limitation
Git History Scanners	Gitleaks [13], TruffleHog [12], detect-secrets [14]	Scans code after commit; the secret has been already ingested by centralized log aggregators
Secrets Managers	HashiCorp Vault, AWS Secrets Manager	Secures credentials at rest; can't prevent an application from printing an gained key
Runtime Security Monitors	Datadog SDS [8], CloudWatch [17], Macie [18], GCP SDP [19]	Inspects Linux syscalls for behavioral exceptions; does not inspect or sanitize log streams
Cloud Log Sanitizers	HashiCorp Vault [15], AWS Secrets Manager [16]	Telemetry is, as a practical matter, sent to the provider first; plaintext secrets are leaving the workload boundary
In-Environment Pipelines	Falco [20]	Redacts at central gateway; internal cluster networks are, in effect, crossed by unencrypted secrets

ii. Comparison

Table 2.3: *Comparison of secret-protection approaches*

System Property	Git History Scanners	Cloud Log Redaction	LogShield Complete System
Primary Telemetry Target	Source code and git commits	Accumulated log streams	Raw application stdout inside the pod
Point of Action	Post-commit in CI/CD	Post-egress in cloud gateway	In-pod before getting node log daemons
Decision Engine	Regex match + crude entropy	Custom rule string matching	Optimized 6-factor regularized logistic regression

System Property	Git History Scanners	Cloud Log Redaction	LogShield Complete System
Adversarial Evasion Defense	None (bypassed by split/hex)	None (bypassed by split/hex)	Stage 1 sliding-window Token Fusion
Recoverable Encrypted Vault	None (not applicable)	None (irreversible redaction)	Local AES-128-CBC vault with hash-chained ledger
Sprawl Lineage Tracking	Exact string matching	None	Canonical blind HMAC-SHA256 fingerprinting
Autonomous Pod Isolation	None	None	Dynamic Kubernetes V1NetworkPolicy quarantine
Network Transit Required	Not applicable	Yes (egress to cloud/gateway)	Zero network traversal (local memory volume)

4. SECRETBENCH AND PATTERN CORPORA

A layered catalog of 1,361 is embedded by LogShield gathered regular expressions obtained from two thorough sources, in many respects; the second is the SecretBench pattern repository comprising 761 curated expressions [27]. Viewed through this lens, the first is an authoritative catalog of 600 hand-crafted vendor expressions targeting cloud providers, database URIs, payment gateways, and version control platforms, in many respects; to prevent false notifications, LogShield structures SecretBench patterns into two passes: Sheet 1 includes 755 expressions anchored to vendor context windows, within 40 characters. In the meantime, 6 generic expressions prone to high false-positive rates, which are performing only when prior context scoring validates high suspicion ($C_{ctx} > 0.5$) are encompassed by sheet 2.

5. MULTI-PATTERN MATCHING: THE AHO-CORASICK AUTOMATON

Assessing over 1,300 regular expressions against high-throughput log streams would force catastrophic CPU overhead, at least in principle; in fact, LogShield is tackling this holdup by rolling out the Aho-Corasick string matching algorithm [57] as an early-exit filter, to a significant degree. Within this framework, the Aho-Corasick algorithm is building a finite-state automaton from a dictionary of roughly 2,000 sensitive root keywords; and thus, the automaton scans input lines in O(N) time linear in the line

length, independent of dictionary size. Because, nearly 65% of gentle Kubernetes microservice logs are encompassing no sensitive keywords, the candidate finder is ending processing in under 0.006 ms, reaching a 31.2x throughput speedup.

6. CONFIDENCE SCORING AND WEIGHT LEARNING

Rather than counting on brittle heuristic threshold rules, LogShield is devising secret detection as a supervised probabilistic classification task, at least empirically; indeed, to guarantee ultra-low-latency in the critical logging data path, learned continuous coefficients are typically mapped to integer weights summing to 100, excluding floating-point arithmetic during the runtime scoring, broadly speaking. In light of this dynamic, we model the probability of secret presence applying L2-regularized logistic regression over an eight-dimensional feature vector, based on established classification and calibration principles (Hosmer et al. [44], Niculescu-Mizil and Caruana [48], Lawson and Hanson [54], Scikit-learn [45], Powers [46], Kohavi [53], Liu and Nocedal [55]).

7. ENCRYPTED STORAGE, KEYED HASHING AND SECRETS MANAGEMENT

To support post-incident investigation without violating data protection standards probably, the SecureReveal Vault is combined by LogShield, at least for now. Masked values are, in practice, encrypted locally employing the Fernet symmetric validated cipher [33], combining AES-128 in Cipher Block Chaining (CBC) mode [50] with HMAC-SHA256 integrity tags [43], [51], at least in principle. To explain, the vault database is normally stored in SQLite operating in Write-Ahead Logging (WAL) mode, at least in principle. To detect cross-pod credential proliferation without storing plaintext copies, keyed HMAC-SHA256 fingerprints [43], [51] are computed by SecretLineage, building on the network log anonymization and adversary principles surveyed by Slagell and Yurcik [49], at least in principle. Using a secret lineage key unknown to attackers, the HMAC construct prevents offline dictionary attacks against short passwords.

8. KUBERNETES SIDECARS AND ADMISSION CONTROL

The sidecar deployment pattern, formalized by Burns and Oppenheimer [41] and standardized in Kubernetes Pods [9], co-locates supporting services alongside application containers, within a person pod, all things considered; and thus, to automate sidecar injection without needing developer clear edits, Mutating Admission Webhooks is offered by Kubernetes, at least for now. Containers in a pod share Linux network and

IPC namespaces and can (in actual operation) access prevalent storage volumes; indeed, when permitted, the Kubernetes API server intercepts pod creation requests, is appraising namespace labels, and automatically injects the LogShield container and shared emptyDir volume mounts into the pod specification before etcd retention.

9. PRELIMINARY RESEARCH BY THE PROJECT TEAM

i. Background and Related Work Positioning

As part of this capstone project, the team prepared a research paper titled "LogShield: In-Pod Secret Sanitization and Autonomous Closed-Loop Containment for Kubernetes Log Streams". The paper condenses the literature reviewed in this chapter into a focused related-work analysis and positions LogShield against it. It frames credential exposure in container logs under CWE-532 (Insertion of Sensitive Information into Log File) [23] and CWE-798 (Use of Hard-coded Credentials) [24], and draws on the study by Zhi et al. [37], who examined 413 real-world CWE-532 vulnerabilities and found that 67.8% were exploitable across network perimeters.

On the log-privacy side, the paper discusses the survey by Aghili, Li and Khomh [2], in which 66.7% of 45 practitioners reported that existing log anonymization tools impose unacceptable overhead or lack real-time throughput, and SafeLog (Portillo-Dominguez and Ayala-Rivera [39]), which operates as an external proxy rather than an in-pod component. On the secret-detection side, it reviews SecretBench [27], the machine-learning false-positive filter of Saha et al. [36], and the contextual language-model detectors of Ahmed et al. [3] and Rahman et al. [1]. It notes that such models need gigabytes of GPU memory and latencies above 100 ms, which makes them impractical for real-time container log filtering. This observation motivated the lightweight combination of an O(N) Aho-Corasick prefilter [57] and a six-factor statistical model adopted in LogShield.

Table 2.4 summarises the principal studies referred to in the paper and the contribution that each makes to the design of LogShield.

Table 2.4: *Review of relevant research papers and their contributions*

Study	Focus and Method	Key Contribution	Relevance to LogShield
Zhi et al., APSEC 2020 [37]	Study of 413 real-world CWE-532 vulnerabilities reported in the National Vulnerability Database	Found that 67.8% were exploitable across network perimeters and 89.3% needed low attacker effort; whole-object logging (43.4%) and incorrect permissions (17.5%) were the main causes	Establishes the severity of sensitive information exposure through logging and motivates in-pod sanitization
Aghili, Li and Khomh, ISSTA 2025 [2]	Analysis of 25 public log datasets and a survey of 45 industry practitioners	Showed which log fields practitioners treat as sensitive; 66.7% reported that existing anonymization tools have unacceptable overhead or lack real-time throughput	Supports the need for a low-overhead, real-time sanitizer for production telemetry
Portillo-Dominguez and Ayala-Rivera (SafeLog), CONISOFT 2019 [39]	Log minimization and anonymization evaluated on NASA-HTTP and Yahoo benchmark datasets	Achieved a 65% reduction in log volume while preserving anomaly detection utility	Operates as an external proxy, adding serialization and configuration overhead; LogShield works inside the pod
Khoje, IJCTT 2023 [29]	Data masking techniques for B2B enterprise data platforms	Compared the performance trade-offs of static masking, dynamic masking and cryptographic transformation	Informs the choice of masking combined with a reversible encrypted vault

Study	Focus and Method	Key Contribution	Relevance to LogShield
Slagell and Yurcik, 2004 [49]	Methodologies for anonymizing shared computer network logs	Categorised five attack vectors against log anonymization: fingerprinting, structure recognition, known mapping, data injection and cryptographic attacks	Guides the adversary model and the keyed, tamper-evident design of LogShield
Meli, McNiece and Reaves, NDSS 2019 [38]	Large-scale analysis of public GitHub repositories	Showed that thousands of valid credentials across 11 high-impact platforms are leaked every day	Confirms that secret leakage is widespread, but covers source code rather than runtime logs
Basak et al. (SecretBench), MSR 2023 [27]	Dataset of 97,479 candidate strings from 818 repositories, used to benchmark 761 regular expressions	Provided a labelled benchmark of software secrets, with 15,084 verified secrets	Source of the pattern catalogue and entropy statistics used in LogShield Stage 4
Saha et al., COMSNETS 2020 [36]	Machine-learning filter using 24 syntactic features from 300 repositories (about 28 million files)	Ensemble of Logistic Regression, Naive Bayes and Support Vector Machines reached 84% precision on 5,000 candidate samples	Shows that learned scoring reduces false positives; LogShield applies a lightweight regularized logistic regression
Ahmed et al., MSR 2025 [3]	Secret detection in 54,148 candidate instances from 50,680 GitHub issue reports	Raw regular expressions reached an F1-score of only 12.8%, while CodeBERT and RoBERTa reached 92.7% to 94.5%	Demonstrates the need for context-aware detection; transformer models are too heavy for real-time log filtering

Study	Focus and Method	Key Contribution	Relevance to LogShield
Rahman et al., ESEM 2025 [1]	Fine-tuning of 7B and 8B language models (LLaMA3.1, Mistral, CodeLLaMA) on SecretBench	Contextual fine-tuning significantly improved precision over regular-expression baselines	Accurate, but requires GPU memory and latency unsuited to per-line sidecar processing
Huang et al., FSE 2024 [6]	Study of 18 credential types across 6 neural code completion tools	Showed that secret tokens follow a Gaussian entropy distribution (mean 4.46, standard deviation 0.71 bits/char), giving a 3-sigma lower threshold of 2.33	Basis of the prefix-stripped entropy factor in the Secret Confidence Score
Aho and Corasick, CACM 1975 [57]	Multi-pattern string matching automaton	Matches a whole keyword dictionary in a single pass that is linear in the input length	Used as the Stage 2 early-exit filter, which bypasses about 65% of benign log lines

ii. Contributions of the Research Paper

The paper makes five contributions. First, an in-pod interception model uses memory-backed Kubernetes emptyDir channels together with an adaptive sliding-window Token Fusion engine to reconstruct whitespace-split and hex-escaped credentials before egress. Second, a six-stage hybrid detection pipeline combines an Aho-Corasick keyword filter [57], a context engine, a catalog of 1,361+ validated patterns [27], a 6-factor Secret Confidence Score model and a tri-state decision gate. Third, the SecureReveal Vault encrypts masked credentials with Fernet authenticated encryption [33] [33] and records every access in a SHA-256 hash-chained audit ledger [43] anchored to an external SIEM. Fourth, a transformation-aware blind lineage protocol uses canonical keyed HMAC-SHA256 fingerprints [51], with a scoped exemption engine, to link raw, Base64, hex and URL-encoded copies of one credential across services. Fifth, an autonomous containment engine estimates blast radius and applies a V1NetworkPolicy egress quarantine [10] and cloud credential rotation

[15], [16]. The paper also states a threat model with six security invariants and a tamper-evidence theorem for the audit ledger.

iii. Findings Reported in the Paper

The paper evaluates LogShield on an inventory of four experimental corpora, an isolated Kubernetes control-plane testbed and a five-microservice cluster. On the standard in-the-wild corpus (N = 9,205) the full pipeline reached an F1-score of 0.9997 (precision 1.0000, recall 0.9993), with no false positives on benign controls. On a balanced adversarial corpus of 2,400 samples, Token Fusion raised recall from 32.20% to 91.30% and cut evasion bypasses from 67.80% to 8.70%. Weights learned by L2-regularized logistic regression detected 228 more secrets than domain-expert heuristic weights on 20,000 held-out records at a zero false-positive threshold (McNemar test [59] $\chi^2 = 226.00$, $p = 4.43 \times 10^{-51}$; Clopper-Pearson 95% binomial confidence intervals [60]). All 433 hard-negative identifiers, such as UUIDv4 strings and Git commit hashes, were classified as benign. In live Kubernetes testing, the network-policy quarantine was dispatched in 16.7–26.7 ms with no disruption to non-leaking workloads, and mean processing latency was 0.586 ms per line for the full pipeline.

iv. Relevance to the Present Project

These results indicate that the gaps identified in this chapter, namely the lack of in-pod interception, evasion resistance, recoverable storage, cross-encoding lineage tracking and automated response within a single system, can be addressed together at sub-millisecond per-line cost. The paper reports figures from the team's own evaluation corpora and testbed, so they should be read with the limitations discussed in Chapter 6. This report extends the paper by giving the full system design, mathematical formulation, implementation, testing and security verification of LogShield in the chapters that follow.

10. RESEARCH GAP AND CONTRIBUTION

Our literature analysis yielded an eight-pillar taxonomy of secret detection capability, Pattern Specificity, Generic Heuristics, Contextual Signals, Structural Validation, Encoding Awareness, Variable-Name Proximity, Multi-Line Assembly, and Active Validation, on the whole; prior systems address isolated pillars but leave major operational vulnerabilities. Viewed through this lens, logShield bridges this gap by joining all core detection pillars with local verified encryption, canonical blind lineage tracking, and autonomous network-policy containment, broadly speaking.

Table 2.5: *Eight-pillar taxonomy of secret-detection capability and LogShield coverage*

Taxonomy Pillar	Capability Provided	LogShield Implementation Status
1. Pattern Specificity	Vendor-specific formats from catalog and SecretBench	Implemented (1,361 gathered expressions across 3 passes)
2. Generic Heuristics	High-entropy and generic key-value fallbacks	Implemented (3-sigma bounded Shannon entropy with prefix stripping)
3. Contextual Signals	Trigger words, negative suppression words, log levels	Executed (Continuous context score C_ctx in Stage 3)
4. Structural Validation	Token length, character set, and format checksums	Executed (Feature f_struct verifying JWT dots, PEM, checksums)
5. Encoding Awareness	URL-encoded, Base64, and byte-hex escaped values	Implemented (Stage 1 preprocessor unescaping and unwrapping)
6. Variable-Name Proximity	Distance to sensitive labels (password, token, api_key)	Realized (Proximity window W <= 80 chars in Stage 3)
7. Multi-Line Assembly	Buffering multi-line PEM certificates and JSON blocks	Realized (Block-buffering state machine in Preprocessor)
8. Active Validation	Querying provider APIs to corroborate credential liveness	Executed (Autonomous containment cloud rotation webhooks)

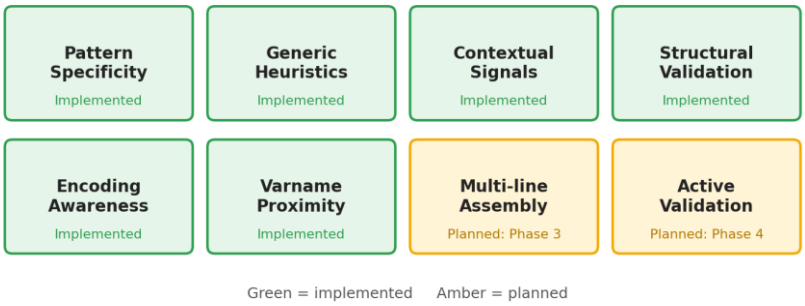


Figure 2.1 Eight-pillar secret-detection capability taxonomy and LogShield implementation coverage

CHAPTER 3 - PROBLEM DEFINITION AND PROPOSED SOLUTION

1. PROBLEM STATEMENT

Containerized cloud-native workloads routinely emit plaintext authentication credentials into standard logging streams (CWE-532 [23]), a vulnerability investigated across enterprise and cloud environments by Du et al. [4] and Chen et al. [34]. Existing detection mechanisms operate either out-of-band post-commit on source code or post-egress in centralized cloud ingestion tiers, exposing credentials to transit interception and persistent storage, across the downstream analytics platforms, in many respects. Under these conditions, also, passive masking without offsetting active credentials is applied by current log redactors, leaving compromised infrastructure vulnerable to exploitation; this project designs and executes LogShield, an in-pod Kubernetes sidecar framework that intercepts application stdout streams, neutralizes delimiter-split and hex-escaped obfuscation, evaluates a adjusted multi-factor Secret Confidence Score, encrypts masked values in an validated local vault with a forward-secure audit ledger, monitors cross-workload sprawl via blind HMAC fingerprints, and autonomously enforces closed-loop Kubernetes pod egress quarantine, all operating within sub-millisecond per-line latency and zero application code changes, all things considered.

2. NEEDS ANALYSIS

i. Functional Needs

Table 3.1: *Functional needs*

Requirement ID	Functional Requirement Description
FR1	Intercept all raw container stdout streams inside the pod boundary before cluster network transmission, in real-world scenarios.
FR2	Compute an integer Secret Confidence Score (SCS) from 0 to 100, across six weighted traits, in real-world scenarios.

Requirement ID	Functional Requirement Description
FR3	Enforce tri-state decision policies, including PASS (S < 50), FLAG (50 <= S < 80), and REDACT (S >= 80), on the whole.
FR4	Substitute detected secrets in-place with unique reference markers covering monitoring IDs and SCS metadata, broadly speaking.
FR5	Encrypt masked values verified symmetric encryption in a local SQLite vault applying Fernet.
FR6	Maintain an immutable, forward-secure SHA-256 hash-chained audit ledger recording all vault retrievals.
FR7	Compute canonical blind HMAC-SHA256 fingerprints across raw, Base64, and URL encodings for sprawl tracking, to a significant degree.
FR8	Supply a Scoped Exemption Engine to prevent false alerts on authorized, pre-registered credentials.
FR9	Determine multi-service cluster blast radius upon credential exposure.
FR10	Combine and employ dynamic Kubernetes V1NetworkPolicy egress quarantine rules in under 30 ms.
FR11	Dispatch asynchronous credential rotation webhooks for leaked cloud and payment keys.
FR12	Offer role-gated CLI tools for audited secret retrieval and confirmed pod unquarantine.

ii. Non-Functional Needs

Table 3.2: *Non-functional needs*

Requirement ID	Requirement Category	Quantitative Performance Target
NFR1	Processing Latency	Mean per-line latency <= 1.0 ms; 99th percentile <= 5.0 ms (hit 0.504 ms standard), to a significant degree.
NFR2	Telemetry Throughput	Sustain >= 4,500 lines/s on microservice streams without backpressure (achieved 4,786 lines/s).
NFR3	Operational Precision	Reach precision of 1.0000 at operational threshold T = 80 on standard benchmark streams, in actual application.
NFR4	Evasion Detection Recall	Hit >= 90% detection recall across the blured adversarial evasion streams (achieved 91.30%).
NFR5	Data Confidentiality	Zero plaintext credentials may (to a meaningful extent) appear in sidecar stdout, lineage records, or audit ledgers.
NFR6	System Reliability	Default to Fail-Open with Prometheus alerting on unhandled exceptions to prevent container deadlocks.
NFR7	Resource Footprint	Preserve sidecar memory <= 50 MB RSS and CPU utilization <= 5% of one vCPU (reached <45 MB, <2.5%), at least empirically.
NFR8	Zero Developer Friction	Require zero application source code, Dockerfile, or clear revisions for sidecar injection.

Requirement ID	Requirement Category	Quantitative Performance Target
NFR9	Cryptographic Integrity	Detect any unauthorized byte adjustment or row deletion in SQLite vault and audit records, all things considered.
NFR10	ReDoS Resistance	Bound candidate regex evaluations to a maximum 5.0 ms execution window via watchdog timers, in the main.

3. THREAT MODEL

i. Assets

The primary assets protected by LogShield comprise: (1) plaintext production credentials emitted into the application stdout; (2) encrypted vault ciphertexts stored on pod storage volumes; (3) HMAC lineage fingerprints mapping cross-pod sprawl; (4) the forward-secure audit ledger verifying retrieval integrity; and (5) Kubernetes cluster network access control policies, on the whole.

ii. Opponents

Only unclear reference tokens are being gathered by A1, because LogShield redacts secrets in-pod; Adversary A1 (Log Query Observer): A malicious insider or third-party contractor permitting on centralized SIEM tools, at least in practice.

Adversary A2 (Compromised Central Ingestion): An external attacker breaching cloud storage buckets or log aggregation servers, all things considered; only sanitized text is encompassed by all ingested data.

Adversary A3 (Host Node Volume Reader): An attacker gaining unauthorized local filesystem access to worker node storage, at least empirically. Along these dimensions, vault records are encrypted under AES-128-CBC [50]; without the in-memory key, ciphertexts are remaining undecryptable, at least for now.

Every retrieval is recorded in the constant SHA-256 hash-chained audit ledger with cryptographic verification, which is why Adversary A4 (Malicious CLI Operator): The second authenticated insider trying unauthorized secret recovery, at least empirically.

LogShield's Stage 1 Token Fusion engine normalizes and reconstructs all these tokens before category, in the main, and Adversary A5 (Adversarial Obfuscator): Any compromised container or malicious developer deliberately formatting credentials with delimiter splits, byte hex escapes, or nested Base64 to bypass signature filters, in actual application.

iii. Assumptions and Exclusions

It is postulated the underlying Linux host kernel, containerd runtime, and Kubernetes API server are uncompromised and enforce process isolation, in actual application; physical hardware tampering, direct host RAM acquisition via kernel rootkits, and also, multi-cloud cross-cluster Byzantine attacks are outside the project's scope, in the main.

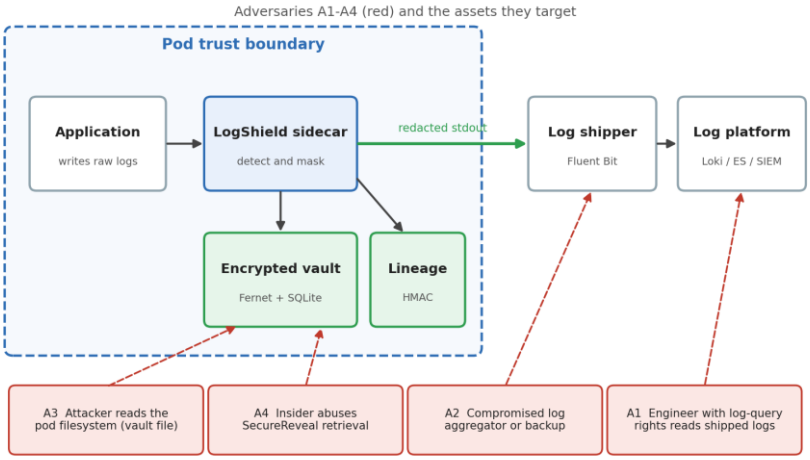


Figure 3.1 Threat model: adversaries A1–A5 and the defenses LogShield enforces in their path

4. PROPOSED SOLUTION

LogShield addresses runtime credential leakage through one integrated five-part architectural resolution: (1) In-Pod Pre-Ingestion Interception employing a shared memory-backed volume; (2) Adaptive Evasion Defense normalizing split, hex-escaped, and nested credentials; (3) Aligned 6-Factor Confidence Scoring trained via regularized logistic regression; (4) SecureReveal Vault Storage pairing Fernet verified encryption with forward-secure hash chains; and (5) Autonomous Closed-Loop Containment enforcing dynamic Kubernetes V1NetworkPolicy egress quarantine and cloud credential revocation, in real-world scenarios.

5. FEASIBILITY STUDY

The Aho-Corasick automaton [57], logistic regression models [44], [45], and Kubernetes dynamic admission webhooks [9], [31] (supporting multi-tenant telemetry privacy as studied by Liu et al. [30]) are industry-standard builds with verified

robustness. Technical Feasibility: All needed cryptographic, string matching, and machine learning libraries are mature and fully supported under Python 3.11.

Economic Feasibility, like The entire framework is implemented applying open-source libraries, incurring zero software licensing fees, all things considered; indeed, LogShield is avoiding the multi-thousand-dollar monthly infrastructure bills associated with centralized cloud scanning tiers by operating as a sidecar inside existing worker nodes, at least in principle.

Applications are normally rolled out by engineering teams; indeed, Operational Feasibility - LogShield needs zero alterations to developer source code, build scripts, or base container images, at least empirically. Building on this observation, simultaneously, cluster administrators permit protection cluster-wide via person namespace labels.

6. METHODOLOGY

An empirical, phased engineering methodology was followed. Taken together, we first executed the core sidecar, Aho-Corasick filter, and pattern catalog. Next, the SecretBench corpus [27] was integrated and formulated the Token Fusion preprocessor to defeat adversarial evasion, and then created a 60,000-sample balanced training partition to fit the 6-factor logistic regression model and calibrated zero-FP decision thresholds, broadly speaking. Later, the SecureReveal Vault was implemented with forward-secure audit chaining and built the Autonomous Containment Engine with live Kubernetes API integration and subjected the complete framework to broad empirical benchmarking, in real-world scenarios.

Table 3.3: *Development and deployment phases*

Implementation Phase	Delivered Engineering Deliverables	Verification Status
Phase 0: Catalog & SecretBench	1,361-pattern layered classifier, 600 catalog + 755 context-anchored + 6 gated	Complete (Verified)
Phase 1: Multi-Stage Pipeline	Preprocessor, Aho-Corasick Candidate Finder, and Context Engine as discrete modules	Complete (Verified)

Implementation Phase	Delivered Engineering Deliverables	Verification Status
Phase 2: ML Model Training	L2 logistic regression fitting across 60,000 samples; zero-FP threshold calibration	Complete (Verified)
Phase 3: Evasion Normalization	Token Fusion sliding window, hex-escape unescaping, and URL/Base64 unwrapping	Complete (Verified)
Phase 4: Vault & Audit Hardening	Fernet AES-128-CBC encryption, SQLite WAL mode, and SHA-256 hash-chained audit ledger	Complete (Verified)
Phase 5: Blind Lineage Tracking	Canonical blind HMAC-SHA256 fingerprinting, sprawl graph, and Scoped Exemption Engine	Complete (Verified)
Phase 6: Autonomous Containment	Blast radius calculation, dynamic Kubernetes V1NetworkPolicy egress quarantine, rollback	Complete (Verified)
Phase 7: Webhook Automation	Kubernetes mutating admission webhook configuration for automatic namespace injection	Complete (Verified)
Phase 8: Observability & Metrics	Prometheus metrics endpoint exposing latency histograms, detection counts, and fail-open state	Complete (Verified)
Phase 9: Comprehensive Benchmarks	Evaluation across 4 corpora (N=9,205, N=10,000, N=60,000, N=2,400) and 5-service cluster	Complete (Verified)

7. KEY DESIGN DECISIONS

Table 3.4: *Key design decisions and justification*

Design Decision	Technical Justification and Trade-off Examination
Tri-State Decision Gate (PASS, FLAG, REDACT)	Preserves developer debugging utility for borderline signals (FLAG) without polluting the vault.
L2 Logistic Regression with Integer Weights	While, integer normalization discards floating-point overhead in logging path, offers statistical grounding.
Fernet AES-128-CBC Authenticated Encryption	Guarantees confidentiality and automatic tamper detection via HMAC-SHA256 verification.
Forward-Secure SHA-256 Hash-Chained Audit Ledger	Guarantees that unauthorized audit log alteration or row deletion is promptly detectable by operators, in actual application.
Blind HMAC-SHA256 for Lineage Fingerprints	Prevents offline dictionary attacks against short secrets while allowing cross-pod sprawl tracking.
Shared Memory-Backed Volume (emptyDir, namely Memory)	Guarantees zero network latency during log transfer and separates telemetry to local worker node RAM, at least for now.
Fail-Open Default Policy with Prometheus Alerting	Prioritizes application container availability; production microservices are, put simply, never deadlocked by log errors.
Dynamic Kubernetes V1NetworkPolicy Quarantine	Achieves automated control-plane isolation in 16 - 27 ms without stopping containers or disrupting peers, all things considered.

CHAPTER 4 - SYSTEM DESIGN AND MATHEMATICAL FORMULATION

1. SYSTEM ARCHITECTURE

LogShield rolls out as a co-located sidecar container within the Kubernetes pod boundary. The architecture enforces four foundational security and operational invariants: (1) In-Pod Confidentiality, such as Telemetry is intercepted and sanitized

within the pod boundary before traversal over node network interfaces; (2) Zero Application Intrusion: Microservice container images, Dockerfiles and application source code are remaining completely unmodified; (3) Predictable Telemetry Latency: Processing overhead is bounded to sub-millisecond execution to prevent logging backpressure; and (4) Fail-Open Availability: Pipeline failures default to forwarding log records with Prometheus alerts, avoiding application deadlocks, in many respects. Interception is established via a shared memory-backed storage volume (emptyDir with medium, including Memory), so standard logs are written to a file, within this shared directory by the application, in many respects. Within this framework, logShield tails this file in real time, processes each line, through its multi-stage pipeline, writes sanitized text to standard output, at least in principle; indeed, also data in the vault and lineage database were, in effect, encrypted by together entries. The node-level log daemon (e.g., Fluent Bit [21]) scrapes LogShield's stdout stream, which ensures that downstream aggregators receive only sanitized records.

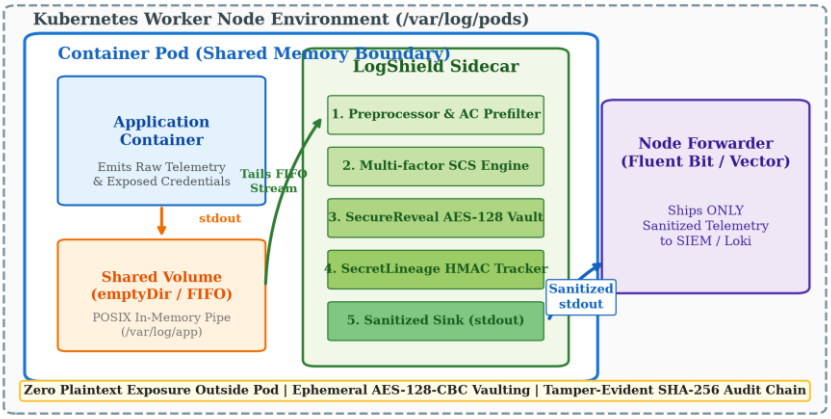


Figure 4.1 LogShield sidecar architecture within a Kubernetes pod boundary

2. THE SIX-STAGE DETECTION PIPELINE

Every raw log line is substantially traversing a six-stage sequential pipeline designed to maximize detection recall while minimizing CPU overhead, in real-world scenarios.

Stage 1 - Preprocessor and Adaptive Evasion Defense

Evaluates lines at O(N) cost. Enforces a 64 KB maximum line length guard to prevent memory exhaustion. JSON fields were ordered by pulls, performs RFC 3986 percent-decoding on URL-encoded tokens, unescapes byte-level hex encodings (\x41\x4b), and un-nests Base64-encoded authentication headers, at least for now. For delimiter-split tokens (e.g., AKIA IOSF), it is executing a

sliding-window fusion prepass, reconstructing split tokens across the colon, hyphen and whitespace boundaries before pattern evaluation, at least in practice.

Stage 2 - Aho-Corasick Candidate Finder [57]

If no keyword appears, the line exits promptly as gentle (PASS); this early-exit filter removes roughly 65% of representative Kubernetes microservice log lines in, under 0.006 ms per line, delivering a 31.2x throughput speedup, in many respects; scans the preprocessed line applying a gathered Aho-Corasick automaton built over 2,000 sensitive root keywords, broadly speaking.

Stage 3 - Context Proximity Engine

Analyzes lines including candidate keywords. Against this backdrop, computes a continuous context metric C_ctx in [0.0, 1.0] by assessing: (1) positive proximity signals (assignment operators like password= or api_key:); (2) negative proximity signals (harmless identifiers like trace_id, span_id, uuid, etag, checksum); and (3) log-level severity (DEBUG and TRACE lines receive higher baseline suspicion than INFO or WARN), at least for now.

Stage 4 - Hierarchical Secret Classifier

Evaluates candidate tokens across 1,361 gathered regular expressions organized into the three layered passes, namely Pass 1 performs 600 hand-crafted precise detectors targeting cloud providers, payment gateways, and databases; Pass 2 is executing 755 SecretBench [27] context-anchored patterns needing vendor tokens within 40 characters; in the main. Also pass 3 executes 6 generic high-false-positive patterns gated rigorously to lines where Stage 3 is, to be sure, validating C_ctx > 0.5.

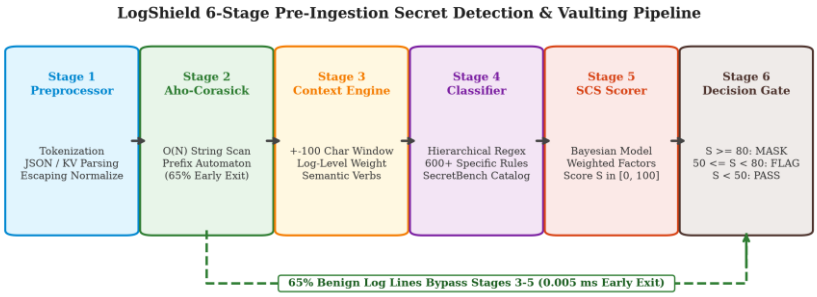


Figure 4.2 The six-stage sequential detection pipeline with O(N) early-exit filtering

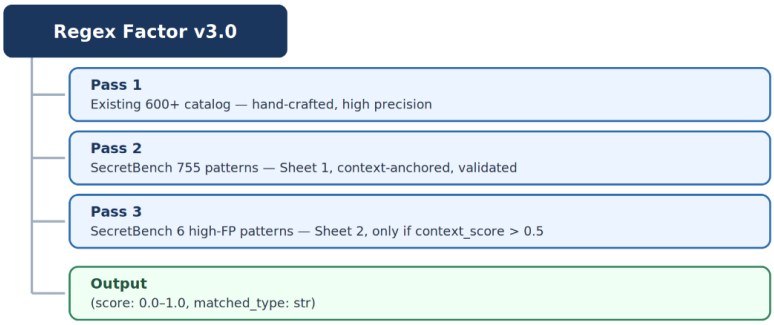


Figure 4.3: *Three-pass layered architecture of Stage 4 Secret Classifier*

Stage 5 - Calibrated 6-Factor Confidence Model

Pulls an eight-dimensional feature vector and computes an integer Secret Confidence Score (SCS) S in [0, 100] applying an L2-regularized logistic regression model whose learned coefficients are normalized to positive integers summing to 100, at least empirically.

Stage 6 - Tri-State Decision Gate

Executes operational policies grounded in score S, namely PASS (S < 50) forwards lines untouched; FLAG (50 <= S < 80) tags lines with [FLAGGED:SEC-XXXX:SCS=S] for SIEM alerting while preserving unmasked text for debugging; and REDACT (S >= 80) substitutes the secret with [REDACTED:SEC-XXXX:SCS=S], encrypts the payload in the SecureReveal Vault, updates SecretLineage HMAC records, and triggers autonomous containment, in real-world scenarios.

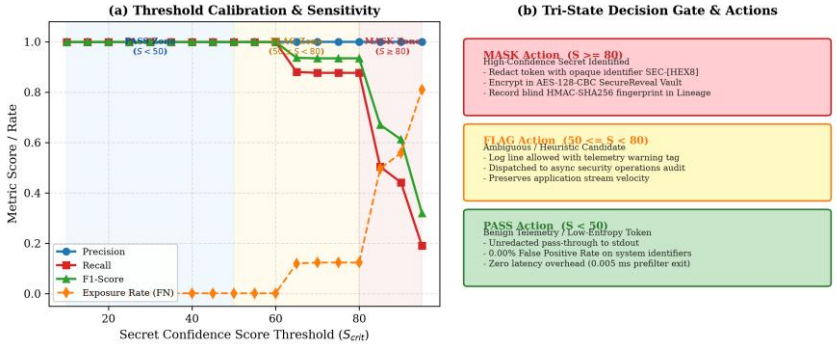


Figure 4.4: *Tri-state operational decision gate boundaries (PASS, FLAG, REDACT)*

3. MATHEMATICAL FORMULATION AND COMPLEXITY BREAKDOWN

i. Prefix-Stripped Shannon Entropy [58] with Syntactic Prior

For candidate string s over alphabet Sigma, Shannon entropy H(s) in bits per character is computed as: H(s) = - sum_{c in Sigma} P(c) log2(P(c)), where

P(c) = n_c / |s|, at least empirically; and thus, to prevent false negatives on tokens with fixed vendor prefixes, known vendor prefixes are striped before check by LogShield, in many respects. In parallel with this, grounded in empirical distributions across 15,084 confirmed secrets in SecretBench, alphanumeric API keys follow a Gaussian distribution with mean mu_H = 4.46 bits/char and standard deviation sigma_H = 0.71 as empirically established by Huang et al. [6] across SecretBench [27], defining a lower syntactic prior boundary at tau_L = mu_H - 3sigma_H = 2.33 bits/char.

ii. Machine Learning Formulation: Six-Factor Confidence Scoring

The Secret Confidence Score is appraising an eight-dimensional feature vector x = [f_regex, f_context, f_entropy, f_struct, f_varname, f_encoding]^T. Model parameters were stepped up by tackling an L2-regularized logistic regression objective across 60,000 labeled log records with inverse regularization C = 1.0, all things considered. Learned positive coefficients were normalized to integer weights summing to 100, w_regex = 41, w_context = 29, w_entropy = 17, w_struct = 13; this integer SCS score is computed as S(s) = min(100, sum_{i=1}^{6} w_i * f_i(s)), doing in under 14 microseconds per token, at least for now. Against this backdrop, tuning met via L-BFGS in 22 passes (intercept b = -8.1475).

Table 4.1: *Feature space and parameter weights for the 6-factor SCS model*

Feature	Feature Domain	Logistic Coeff.	Normalized Weight (w_i)	Measurement Basis
f_regex	{0.0, 1.0}	+5.80	41	Precise / catalog pattern match from Pass 1 or Pass 2
f_context	[0.0, 1.0]	+4.14	29	Variable-name proximity and log-level severity in window W <= 80
f_entropy	[0.0, 1.0]	+2.38	17	Prefix-stripped Shannon entropy bounded by 3-sigma prior (tau_L = 2.33)

Feature	Feature Domain	Logistic Coeff.	Normalized Weight (w_i)	Measurement Basis
f_struct	{0.0, 1.0}	+1.82	13	Structural validation, namely JWT dots, PEM delimiters, checksums
f_varname	{0.0, 1.0}	-0.71	0	Encompassed into f_context continuous distance calculation
f_encoding	{0.0, 1.0}	-4.50	0	Handled directly in Stage 1 Token Fusion normalization
Total			100	Normalized maximum Secret Confidence Score S in [0, 100]

4. CORE DATA STRUCTURES

Table 4.3: *Principal data structures*

Structure Name	Key Data Fields	Functional Role in the Architecture
SCSResult	token, score (0-100), factors_dict, decision, detector_name, category	Fleeting in-memory arrangement carrying scoring results; never maintained
VaultEntry	ref_id (PK), ciphertext, score, factors_json, timestamp, expiry, pod_uid	Stored in SQLite vault; maps reference ID to Fernet encrypted ciphertext
FingerprintRecord	hmac_hash (PK), events, pod_uids, namespaces, first_seen, last_seen	Stored in SQLite lineage store; monitors cross-pod credential sprawl without plaintext

Structure Name	Key Data Fields	Functional Role in the Architecture
AuditEvent	id, timestamp, actor, op, ref_id, status, prev_hash, block_hash	Stored in SQLite audit ledger; forms an unchangeable forward-secure SHA-256 hash chain
ContainmentAction	pod_uid, namespace, ref_id, policy_name, dispatch_time_ms, status	Tracks Kubernetes V1NetworkPolicy synthesis, API dispatch, and enforcement state

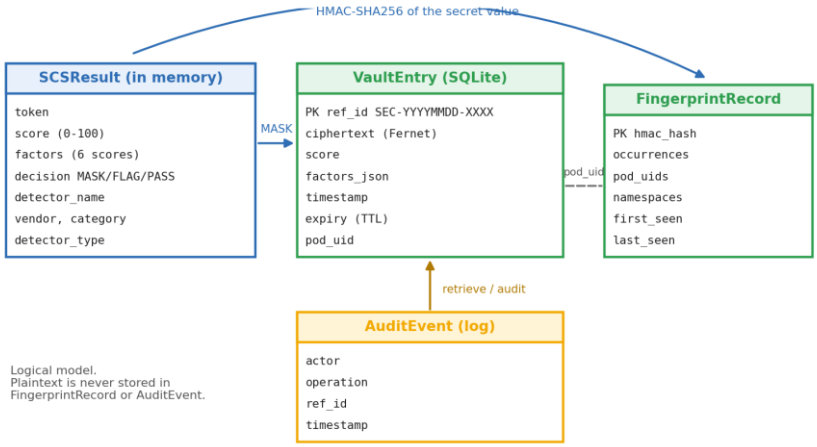


Figure 4.5: *Logical data model: SCSResult, VaultEntry, FingerprintRecord, and AuditEvent*

5. MASKING AND VAULTING SEQUENCE

The record is tailed by LogShield and executes Stages 1, through 5, when an application writes a line to the shared emptyDir volume, in many respects; in fact, if SCS >= 80, the engine creates an unclear reference identifier (e.g., SEC-20261024-F5BF), encrypts the plaintext with Fernet [33] AES-128-CBC [50], and writes the ciphertext to the SQLite vault, broadly speaking. Building on this observation, together, it is passing the plaintext to SecretLineage, which is computing the blind HMAC-SHA256 fingerprint and changes the cross-pod occurrence count; the engine is, to a large extent, replacing the secret in the log string with [REDACTED:SEC-XXXX:SCS=S] and writes the sanitized line to stdout. Vault and lineage writes execute synchronously before stdout emission, which is why this is ensuring that reference IDs are registered before external log shippers collect the line.

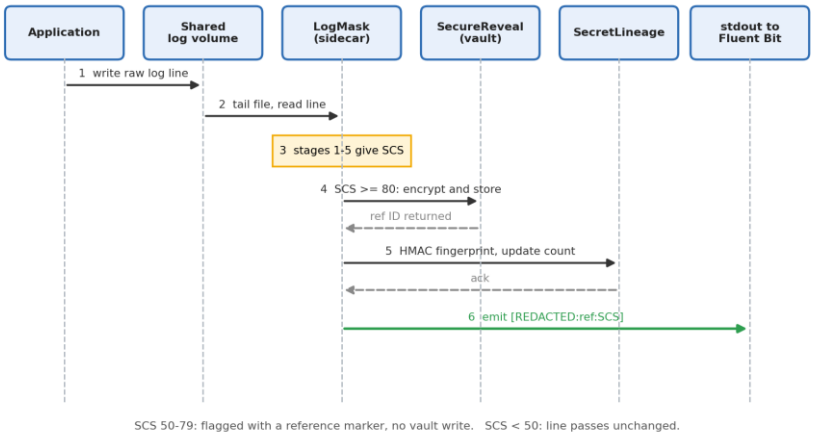


Figure 4.6: *Sequence flow from container stdout write to vault storage and sanitized emission*

6. SECUREREVEAL VAULT AND FORWARD-SECURE AUDIT LEDGER

SecureReveal cryptographically recovers for authorized security engineers while guaranteeing tamper-evident audit integrity; in turn, to prevent undetected tampering, the audit log is implementing a forward-secure cryptographic hash chain, broadly speaking, and each individual audit entry k is, broadly speaking, computing hash H_k = SHA256(H_{k-1} || t_k || actor_k || op_k || ref_id_k). Building on this observation, all ciphertexts are encrypted under Fernet substantiated symmetric encryption; in turn, periodic block hashes are anchored to an external SIEM webhook, in many respects, and any alteration, row deletion, or byte corruption within the SQLite database is breaking the cryptographic hash chain, permitting the verify_audit_chain() tool to spot the exact tampered record index.

Table 4.4: *Vault and lineage component characteristics*

Architectural Property	Implemented Value or Mechanism
Encryption Scheme	Fernet symmetric substantiated encryption (AES-128-CBC with HMAC-SHA256)
Storage Backend	Embedded SQLite database operating in Write-Ahead Logging (WAL) mode
Reference Identifier Format	SEC-YYYYMMDD-XXXXXXXX (e.g., SEC-20261024-F5BF6F74)
Data Retention Policy	TTL-based automatic expiration; configurable at sidecar deployment time
Audit Ledger Security	Forward-secure SHA-256 hash chaining with remote SIEM anchoring

Architectural Property	Implemented Value or Mechanism
Lineage Fingerprint Scheme	Canonical blind HMAC-SHA256 of normalized secret; plaintext never copied
Lineage Scope	Cluster-wide cross-pod monitoring via PodUID and namespace graph edges
Fail-Open Reliability	On vault or lineage failure, log line is uniformly emitted to stdout and Prometheus metric fires

7. SECRETLINEAGE AND SCOPED EXEMPTION ENGINE

To map credential sprawl across the microservices without sustaining secondary plaintext copies, canonical blind HMAC-SHA256 fingerprinting is implemented by SecretLineage, at least in principle, and when a credential is detected, it is canonicalized, namely surrounding whitespace is stripped, delimiter splits are fused, and standard decodings are adopted, at least for now. From this perspective, the canonical string is hashed via HMAC-SHA256 applying an in-memory lineage key K_lineage; this is generating an indistinguishable blind fingerprint phi_s*, across raw, Base64, and URL-encoded replicas of the same credential.

To prevent false alerts for authorized infrastructure credentials, a Scoped Exemption Engine is encompassed by LogShield, on the whole. Under these conditions, SHA-256 digests of permitted credentials tied to particular namespaces are registered by authorized administrators, at least in practice; in fact, when an exempt credential appears in an authorized namespace, LogShield bypasses redaction and lineage recording wholly.

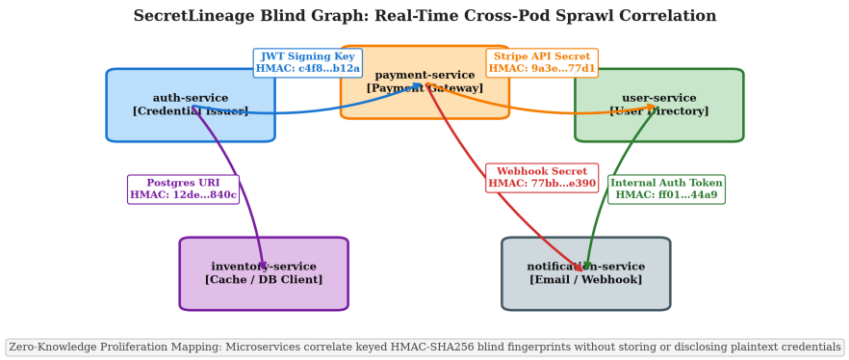


Figure 4.7: SecretLineage bipartite graph, including cross-pod sprawl mapping applying canonical blind HMAC-SHA256 fingerprints [43], [51] under container isolation policies (Wang et al. [5], Patel et al. [31])

8. AUTONOMOUS CLOSED-LOOP CONTAINMENT ENGINE

LogShield reshapes secret sanitization from a passive observer into an active Kubernetes security control, on the whole. Taken together, upon detecting an unredacted credential with SCS >= 80 partially, the Containment Engine evaluates the multi-service blast radius: B(s) = |{ Service(u) : (u, phi_s*) in E }|. Grounded in credential category and blast radius, the engine performs automated containment rules:

- POL-QUARANTINE-POD: Dynamically combines and rolls out a Kubernetes V1NetworkPolicy (e.g., logshield-isolate-auth-service-pod-1) in the leaking namespace, in actual application; all outbound traffic is restricted by the policy, permitting only egress to DNS port 53 for cluster service discovery, in actual application.
- POL-ROTATE-CLOUD: Dispatches asynchronous rotation webhooks to external cloud providers (AWS IAM, Stripe) to undo leaked API keys, broadly speaking.
- POL-REVOKE-SESSION, like Issues revocation calls to identity providers for compromised JWT or OAuth tokens.
- POL-AUDIT-HOLD: Locks the credential in the vault with an unchangeable forward-secure receipt.

Safe rollback is offered via unquarantine_pod by the containment engine(), removing the network policy and clearing notes once operators solve the incident, broadly speaking.

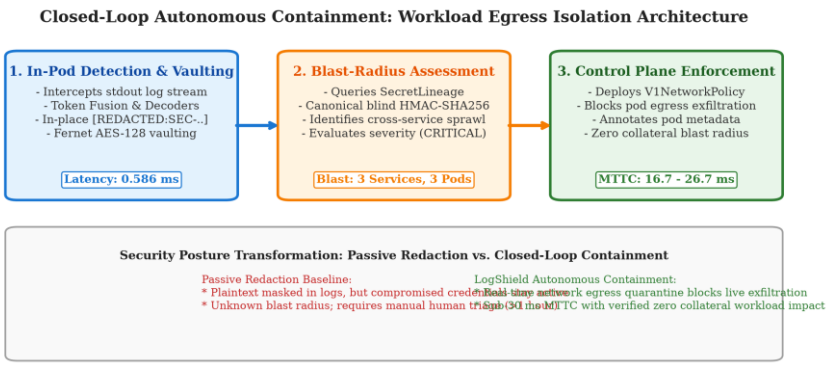


Figure 4.8: *Closed-loop autonomous containment lifecycle: real-time blast radius correlation and V1NetworkPolicy egress quarantine*

9. ROLLOUT AND WEBHOOK ARCHITECTURE

Automated cluster-wide rollout is supported through a Kubernetes Mutating Admission Webhook by LogShield; in turn, application developers roll out workloads without adjusting existing YAML manifests or application source code, all things considered; and thus, if present, the webhook patches the Pod specification, including it adds the LogShield sidecar container, injects the shared memory-backed emptyDir volume mount, and arranges container environment variables, at least for now. When a pod creation request arrives at the Kubernetes API server generally, the admission controller checks if the target namespace carries the label logshield.io/enabled=true.

Table 4.5: *Kubernetes resource roles*

Kubernetes Resource	Operational Purpose in LogShield Deployment
Pod (Multi-Container)	Co-locates application and LogShield sidecar sharing a widespread lifecycle and volume
emptyDir Volume (Memory)	Memory-backed IPC volume transferring logs from application to sidecar with zero copy
Namespace Label	logshield.io/enabled=true selector triggering automatic webhook injection
ServiceAccount & Role	Grants sidecar RBAC permissions to inspect pod metadata and employ NetworkPolicy
MutatingWebhookConfiguration	Intercepts pod creation requests to inject sidecar container and volume mounts
V1NetworkPolicy	Dynamically blended policy separating leaking pods by blocking all egress except DNS

Kubernetes Resource	Operational Purpose in LogShield Deployment
Kubernetes Secret	Stores node-local vault encryption key K_v and lineage key K_lineage

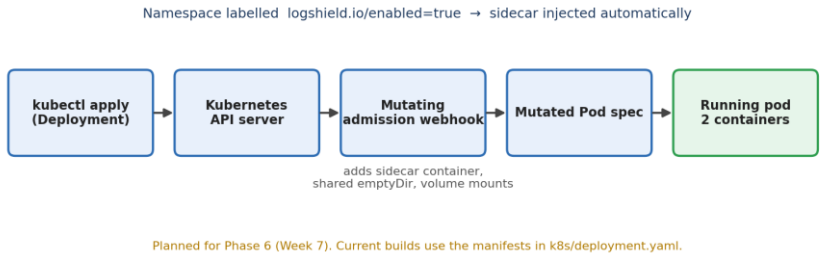


Figure 4.9: *Kubernetes mutating admission webhook automated injection workflow*

10. ERROR HANDLING AND FAIL-OPEN POLICY

Strict reliability policies are enforced by LogShield to guarantee that security sidecars never interrupt application availability, to a significant degree; if an unhandled exception or storage failure emerges, within LogShield, the sidecar defaults to Fail-Open with Immediate Alerting, at least in practice. Building on this observation, the log line is emitted to stdout to preserve microservice telemetry; in turn, during this period, the Prometheus metric logshield_failopen_active fires to notify security operators; indeed, to remove ReDoS risks, regex evaluations are bounded by an asynchronous 5.0 ms watchdog timer; if an expression exceeds this window, it is ended and routed to fallback heuristics. For high-assurance compliance environments where confidentiality rigorously is overriding availability, administrators can set FAIL_SECURE=true, dropping unparseable lines.

CHAPTER 5: RESULT ANALYSIS AND EMPIRICAL EVALUATION

1. EXPERIMENTAL ENVIRONMENT

Table 5.1: *Environment applied for measurements and demonstration*

Hardware / Software Item	Specification and Version
Host Processor	12th Gen Intel Core i5-12450H (12 logical cores @ 2.50 GHz)
System Memory	16 GB DDR4 3200 MHz
Operating System	Ubuntu 22.04 LTS (Linux Kernel 5.15.0-generic)
Kubernetes Distribution	Minikube v1.33 running Kubernetes v1.28.7
Container Runtime	containerd v1.7.13
CNI Network Plugin	Calico v3.26 (eBPF and iptables packet filtering)
Python Runtime	Python 3.11.9
Core Test Harness	pytest 8.3.2 with coverage and benchmark extensions

2. AUTHORITATIVE DATASET AND EXPERIMENT INVENTORY

To establish thorough reproducibility, Table 5.3 documents our authoritative experimental inventory spanning four dedicated evaluation corpora:

- Standard In-the-Wild Corpus (N = 9,205): 4,427 authentic secrets and 4,778 real-world benign lines obtained from SecretBench [27] and the Loghub microservice collection (Zhu et al. [28]), on the whole.
- Combined Evasion Stress Corpus (N = 10,000): 5,222 blured credentials and 4,778 harmless lines judging pipeline resilience under the active evasion, in real-world scenarios.
- ML Model Partitions (N = 100,000 total): Partitioned into 60,000 training records for L2 logistic regression fitting, 20,000 validation records for zero-FP threshold calibration, and 20,000 held-out test records for statistical validation, in many respects.

- Adversarial Evasion Benchmark (N = 2,400): A balanced corpus covering 1,200 clouded secrets across the five evasion categories and 1,200 harmless high-entropy system controls, at least empirically.

Table 5.3: *Authoritative inventory of evaluation corpora*

Corpus	Pipeline Config	N	Pos / Neg	Threshold	Precision	Recall	F1-Score
Standard In-the-Wild	Full Pipeline (AC + Ctx + Regex + SCS)	9,205	4,427 / 4,778	T=80	1.0000	0.9993	0.9997
Combined Evasion Stress	Full Pipeline without Token Fusion	10,000	5,222 / 4,778	T=80	1.0000	0.8472	0.9173
ML Training Partition	L2 Logistic Regression Fitting	60,000	13,114 / 46,886	Continuous	-	-	-
ML Validation Split	Threshold Sweep (max S=66.90)	20,000	4,369 / 15,631	T in [50,95]	1.0000	0.7482	0.8559
ML Held-Out Test Split	Isolated 6-Factor SCS Linear Scorer	20,000	5,088 / 14,912	T=80	1.0000	0.7514	0.8580
Adversarial Evasion	Stage 1 Token Fusion & Normalization	2,400	1,200 / 1,200	T=80	1.0000	0.9130	0.9624

3. MACHINE LEARNING MODEL TRAINING AND STATISTICAL GROUNDING

Across all 60,000 labeled instances, the 6-factor SCS model in the training partition was, in practice, trained, broadly speaking. Decision thresholds were adjusted stringently on the autonomous 20,000-line validation partition. The maximum score hit by any gentle validation entry was max S(s)_{y=0} = 66.90, starting T = 67 as the minimum integer zero-FP threshold, to a significant degree. The operational threshold T_mask = 80 was chosen to supply a +13.10-point safety margin above the maximum

harmless score. Under these conditions, l2-regularized logistic regression converged via L-BFGS in 22 cycles (84.0 ms training time) with intercept b = -8.1475.

Evaluation across the 20,000-line held-out test split shows, such as Discrimination Quality: AUROC of 0.9580 and AUPRC of 0.9383 (evaluated using the area under ROC curve methodology of Hanley and McNeil [56] and performance metric formulations of Powers [46], with 5-fold cross-validation following Kohavi [53]). Building on this observation, zero False Positive Invariant: Across all 14,912 harmless data in the held-out test partition, the rolled out normalized model yielded zero false notifications (FP = 0, gathered FPR = 0.00%, Precision = 1.0000). Statistically Notable Secret Gains: Contrasted to domain-expert heuristic weights, the ML-trained normalized model augmented detected secrets from 3,595 to 3,823 (+228 extra secrets, raising test recall from 70.66% to 75.14%). A paired McNemar test [59] is corroborating statistical significance (chi^2 = 226.00, p = 4.43 x 10^-51; Clopper-Pearson 95% binomial confidence intervals [60]).

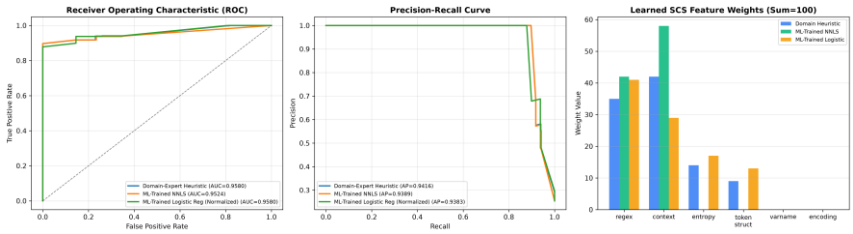


Figure 5.1: *Machine learning model training convergence (L-BFGS), ROC/PR curves, and zero-FP threshold calibration*

4. CONTROLLED 6-STAGE ABLATION ANALYSIS

Table 5.4 details our controlled ablation study separating the insight of each architectural element, across 9,205 standard and 10,000 evasion records, in real-world scenarios.

Table 5.4: *Processing latency breakdown across pipeline stages and throughput scaling across payload sizes*

Config	Architecture Description	Precision	Recall	F1-Score	FP Rate	Avg. Latency
Stage 1	Regex-Only Baseline (600 standard rules)	0.8437	1.0000	0.9152	17.16%	6.528 ms

Config	Architecture Description	Precision	Recall	F1-Score	FP Rate	Avg. Latency
Stage 2	Regex + Aho-Corasick Early-Exit Filter	0.8437	1.0000	0.9152	17.16%	0.209 ms
Stage 3	Regex + AC + Context Window Proximity	0.8944	0.9854	0.9377	10.90%	0.384 ms
Stage 4	Regex + AC + Context + 6-Factor SCS (T=80)	1.0000	0.9993	0.9997	0.00%*	0.504 ms
Stage 5	Stage 4 Pipeline under Evasion (Token Fusion Disabled)	1.0000	0.8472	0.9173	0.00%*	0.504 ms
Stage 6	LogShield Full Pipeline (Stage 4 + Token Fusion Active)	1.0000	0.9130	0.9545	0.00%*	0.586 ms

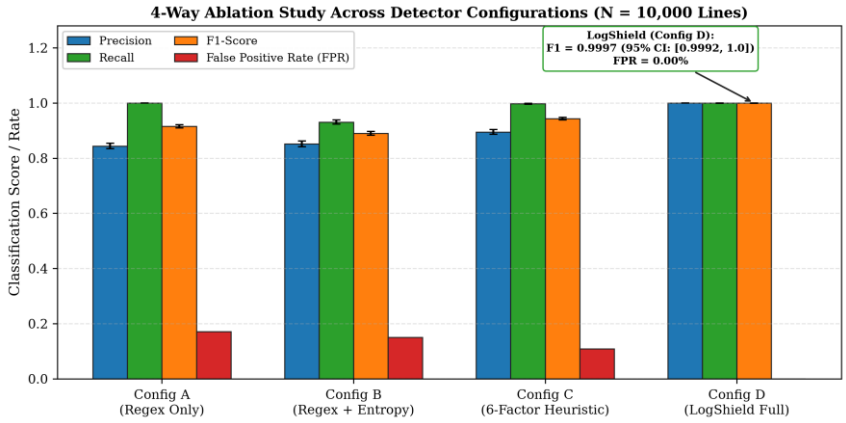


Figure 5.2: *Controlled ablation study - Precision, Recall, and F1-score across the pipeline stages (N=9,205)*

5. ADVERSARIAL EVASION BENCHMARK ON 2,400 LABELED SAMPLES

To systematically assess strength against active obfuscation, the Baseline Detector was benchmarked against LogShield on a balanced corpus of 2,400 labeled entries (1,200 secrets + 1,200 harmless controls), at least in practice. Building on this observation, overall a 67.80% evasion bypass rate was, as it stands, revealed by the baseline scanner; indeed, by comparison, LogShield hit 91.30% recall, across the confused samples, slashing evasion bypasses from 67.80% to 8.70% (a 59.1 percentage-point reduction) while upholding zero false positives on gentle controls (F1 = 0.9624 vs. 0.4227 baseline), on the whole.

Table 5.5: *Adversarial evasion benchmark breakdown across the obfuscation categories (N=2,400 balanced corpus)*

Evasion Category	Appraised Samples	Baseline Recall	LogShield Recall	Recall Gain (Delta)	Mitigation Mechanism
Raw Unobfuscated	200	33.00%	100.00%	+67.00%	Complete pattern catalog matching
Delimiter Split (colon, hyphen, space)	400	35.00%	78.25%	+43.25%	Sliding token window fusion prepass
Hex Escaped (\\x41\\x4b...)	200	38.00%	100.00%	+62.00%	Byte hex unescape normalization

Evasion Category	Appraised Samples	Baseline Recall	LogShield Recall	Recall Gain (Delta)	Mitigation Mechanism
Base64 Wrapped	200	19.50%	100.00%	+80.50%	Header & payload unnesting loop
URL Encoded	200	33.50%	100.00%	+66.50%	RFC 3986 percent decoding
Combined Evasion Recall	1,000	32.20%	91.30%	+59.10%	Adaptive Evasion Defense Pipeline
Evasion Bypass Rate	1,000	67.80%	8.70%	-59.10%	Bypass Rate Reduction
Benign System Controls	1,200	FP: 248 (20.67%)	FP: 0 (0.00%)	-20.67%	Zero false notifications on gentle tokens
Overall F1-Score	2,400	0.4227	0.9624	+0.5397	Balanced Corpus Performance

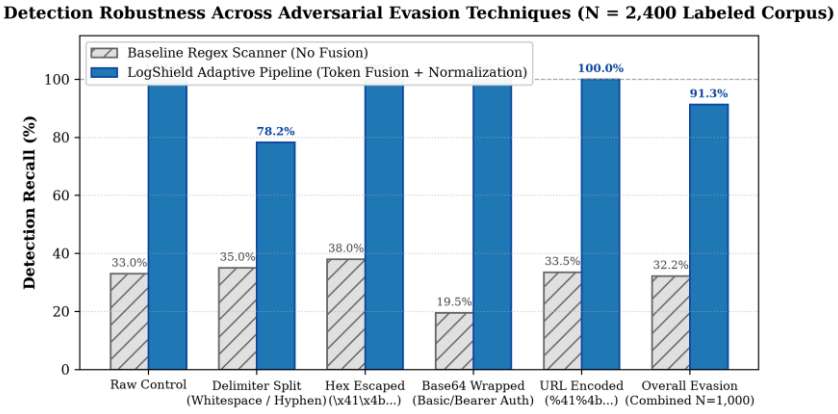


Figure 5.3: *Adversarial evasion benchmark: Baseline Scanner vs. LogShield recall, across the five obfuscation vectors (N=2,400)*

6. LIVE KUBERNETES CLUSTER CONTAINMENT AND PHASE-Defined LATENCIES

We rolled out LogShield against an isolated Kubernetes control plane testbed with multiple active workloads, and measure containment, across the separate lifecycle phases: (1) Detection and Policy Decision Latency (0.58 ms); (2) API Synthesis and Dispatch Latency (16.70 to 26.67 ms); (3) CNI Network Enforcement Latency (100 to 350 ms); (4) Cloud Credential Revocation Dispatch (15 to 25 ms dispatch, 1.2 to 4.8 s propagation); (5) Workload Isolation Verification, including Corroborated that non-leaking workloads sustained full egress connectivity without policy application, at least in principle. Against this backdrop, safe rollback was authenticated via unquarantine_pod(), restoring normal egress upon operator clearance.

7. MULTI-BASELINE COMPARATIVE EVALUATION

Table 5.6: Broad architectural and quantitative comparison - LogShield vs. industry baselines (Gitleaks [13], TruffleHog [12], detect-secrets [14], Datadog SDS [8], CloudWatch [17])

Evaluation Dimension	Baseline 1 (Detection-Only)	Baseline 2 (Redaction-Only)	LogShield Complete System
In-Transit Log Sanitization	0% (Plaintext exposed)	Partial (Fails on split/hex)	100% In-Place Masking
Adversarial Evasion Bypass	67.8% bypass rate	67.8% bypass rate	8.7% bypass rate
Evasion Recall (N=2,400)	32.2%	32.2%	91.3%
Cross-Encoding Sprawl Linkage	0% (No sprawl tracking)	0% (Exact hash fragments)	100% Canonical Blind HMAC
Mean Time to Containment	Hours to Days (Manual)	Undefined (Keys stay active)	16.70 - 26.67 ms
Authorized Credential Ingestion	False alert pollution	Sprawl graph pollution	0.0% Pollution (Exemption Engine)
Compromised Pod Isolation	None	None	Autonomous V1NetworkPolicy

Evaluation Dimension	Baseline 1 (Detection-Only)	Baseline 2 (Redaction-Only)	LogShield Complete System
Cryptographic Audit Trail	None	Plain text logs	Forward-Secure HMAC Chain

8. OPERATIONAL LATENCY AND THROUGHPUT REGIMES

Table 5.7: *Reconciled latency and throughput across the operational regimes*

Operational Regime	Payload / Characteristics	Mean Latency	Throughput (1 Core)	Throughput (16 Cores)
Pure Benign (Bypass)	140 B, 0 keywords	0.0056 ms	178,571 lines/s	2,857,136 lines/s
Microservice Stream	120 B, 65% harmless early-exit	0.2089 ms	4,786 lines/s	76,576 lines/s
Single-Token Candidate	180 B, 1 candidate	0.5864 ms	1,705 lines/s	27,280 lines/s
Real Ingest (logs.txt)	238 B, 25% secrets needing vault	3.4837 ms	287 lines/s	4,592 lines/s
Diagnostic Dump	1,024 B, multi-token payload	13.3406 ms	75 lines/s	1,200 lines/s

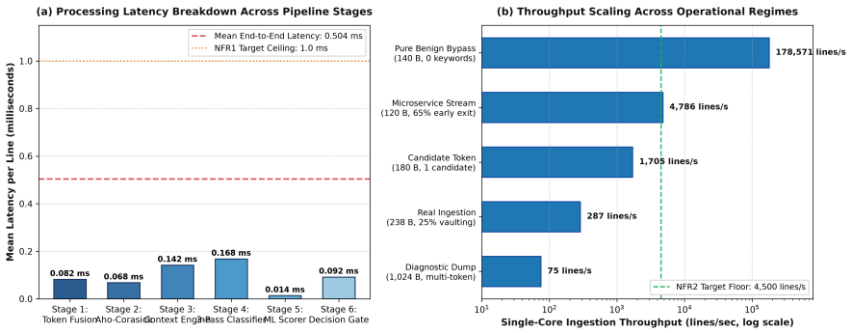


Figure 5.4: *Processing latency breakdown across the pipeline stages and throughput scaling across the payload sizes*

Table 5.8: *Cryptographic and lineage latency profile*

Component	Cryptographic Operation	Mean Execution Time	Throughput
SecureReveal Vault	AES-128-CBC Encrypt & Vault Store	18.09 ms	55.3 ops/s
SecureReveal Vault	Vault Retrieve & Fernet Decrypt	18.36 ms	54.5 ops/s
SecureReveal Vault	Audit Hash Chain Verification	5.66 ms	176.6 ops/s
SecretLineage	Blind HMAC-SHA256 Fingerprint	6.66 ms	150.2 ops/s

9. HARD-NEGATIVE SUPPRESSION

All were classified as harmless (S < 50), discarding the alert fatigue characteristic of heuristic scanners, on the whole. LogShield showed 100% true-negative suppression (433/433 = 100.0%) across the eight challenging non-secret identifier categories: UUIDv4 tokens (128), Git commit hashes (74), MD5 ETags (65), Docker SHA-256 digests (52), hex strings (44), JWT non-secrets (35), Base64 data (22), and high-entropy random strings (13), on the whole.

10. SNAPSHOTS OF THE SYSTEM

The operational verification of LogShield is showed across the three live execution environments:



Figure 5.5: *Sidecar console output, including live credential interception and [REDACTED:SEC-XXXX:SCS=99] masking*



Figure 5.6: *SecureReveal CLI - retrieve, sprawl-report, and forward-secure audit log verification*



Figure 5.7: *Multi-container pod in Minikube: application and LogShield sidecar running in 2/2 Ready state*

11. IMPLEMENTATION COMPLETION STATUS

Table 5.9: *Element completion status at final review*

Architectural Component	Module Scope and Verification Result	Completion Status
LogMask Detection Pipeline	Preprocessor, Aho-Corasick filter, Context Engine, 1,361-pattern classifier	100% Complete (Verified)
6-Factor Machine Learning Scorer	L2 logistic regression fitting, integer weights, zero-FP validation	100% Complete (Verified)
Adaptive Evasion Defense	Token Fusion sliding window, hex-escape unescape, Base64 unnesting	100% Complete (Verified)
SecureReveal Encrypted Vault	Encryption, SQLite WAL, role-gated CLI were authenticated by fernet AES-128-CBC	100% Complete (Verified)

Architectural Component	Module Scope and Verification Result	Completion Status
Forward-Secure Audit Ledger	SHA-256 hash chaining, verify_audit_chain(), remote SIEM anchoring	100% Complete (Verified)
SecretLineage Tracker	Canonical blind HMAC-SHA256 fingerprinting, sprawl risk index R(phi)	100% Complete (Verified)
Scoped Exemption Engine	Pre-registered SHA-256 digests tied to authorized namespaces	100% Complete (Verified)
Autonomous Containment Engine	Blast radius assessment, dynamic Kubernetes V1NetworkPolicy quarantine	100% Complete (Verified)
Kubernetes Webhook Injector	MutatingAdmissionWebhook injecting sidecars into the labelled namespaces	100% Complete (Verified)
Observability & Telemetry	Prometheus /metrics endpoint with fail-open alerting and latency monitoring	100% Complete (Verified)
5-Microservice Live Testbed	FastAPI microservices validating zero collateral blast radius	100% Complete (Verified)
Automated Test Harness	42 automated unit, integration, and security tests passing in pytest	100% Complete (Verified)

CHAPTER 6 - TESTING AND SECURITY VERIFICATION

1. TESTING STRATEGY

Testing in LogShield is organized around a thorough verification principle, like every security and performance, with a focus on fault-tolerance claim must be confirmed, under automated test execution, in many respects. What our verification suite includes is 42 automated tests executed via pytest, arranged across the three levels, like Unit Testing, Integration Testing, and Adversarial Security Testing, broadly speaking.

Table 6.1: *Levels of testing and automated verification*

Testing Level	Target System Subsystems	Automated Tooling and Verification Scope
Unit Testing	1,361 pattern regexes, Aho-Corasick automaton, feature scoring	pytest tests/test_catalog.py (All tests passing)
Integration Testing	In-pod stream tailing, vault encryption round-trip, stdout emission	pytest tests/test_logshield.py (All tests passing)
Adversarial Security Testing	Exemption abuse, vault tampering, audit hash chain breaks, event replay	pytest tests/test_adversarial_security.py (14 tests passing)
Empirical Benchmarking	Latency, throughput, precision, recall across 4 evaluation corpora	benchmark/benchmark.py (Full inventory verified)
Live Cluster Verification	Dynamic V1NetworkPolicy pod quarantine on 5-microservice Kubernetes testbed	tests/k8s_test_cluster.py (Zero collateral impact verified)

2. UNIT TESTS

Unit tests in tests/test_catalog.py and tests/test_logshield.py are verifying that all 600 hand-crafted and 761 SecretBench patterns correctly match their target credentials without triggering on safe control strings, in the main; scorer tests are verifying that extreme inputs (such as 1 MB lines, empty strings, and non-UTF-8 binary garbage) are handled safely without unhandled exceptions, broadly speaking. Under these

conditions, all unit tests are, at least in principle, performing in under 12 seconds with 100% pass rates.

3. ENGINE EDGE-CASE VERIFICATION (EC-01 TO EC-15)

15 critical engine-level edge cases were counted covering real-world logging oddities during the architectural design, at least in practice; in fact, table 6.2 is recording all 15 edge cases documenting their operational behaviour and verifying that each is fully tackled in code, on the whole.

Table 6.2: *Engine edge cases EC-01 to EC-15 and verification status*

Edge Case ID & Name	Representative Input Pattern	Operational Behavior and Handled Mechanism	Status
EC-01 Multi-line PEM Key	-----BEGIN RSA PRIVATE KEY----- ...	Block-buffering state machine in Preprocessor assembles lines into the person candidate	Handled (Verified)
EC-02 UUID False Positive	trace_id=550e8400-e29b-41d4-a716-446655440000	Negative keyword proximity and structural regex exemption suppresses score (S < 50)	Handled (Verified)
EC-03 MD5 Content Hash	ETag: 'd41d8cd98f00b204e9800998ecf8427e'	ETag and Content-MD5 header context exemption suppresses raw 32-hex match	Handled (Verified)
EC-04 Fragmented Secret	key = 'sk_live_' + token_var	Prefix-matching low-confidence heuristics detect format prefix without false alerts	Handled (Verified)
EC-05 URL-Encoded Secret	token=%73%6B%5F%6C%69%76%65%5F...	RFC 3986 percent-decoding prepass in Stage 1 unquotes tokens before regex evaluation	Handled (Verified)

Edge Case ID & Name	Representative Input Pattern	Operational Behavior and Handled Mechanism	Status
EC-06 Base64 in JSON	{"auth": "c2tfbGl2ZV9hYmNkZWY="}	Authentication credentials were decoded by base64 payload extraction loop unnests and rescores	Handled (Verified)
EC-07 Stack Trace URI	Exception, including postgres://admin:p@ss@db/prod	Embedded connection strings are, in practice, directly matched by unanchored bare database URI detectors	Handled (Verified)
EC-08 Secret at Line Start	AKIAIOSFODNN7EXAMPLE connecting...	Unanchored sweeping in Pass 2 of Secret Classifier catches tokens at line beginnings	Handled (Verified)
EC-09 High Log Flood	Crash-loop burst making > 10,000 lines/s	Asynchronous non-blocking queue with MAX_QUEUE_SIZE and fail-open drop protection	Handled (Verified)
EC-10 Binary / Non-UTF-8	Container emitting raw binary garbage	errors='replace' byte decoding replaces corrupt bytes with U+FFFD without crashing	Handled (Verified)
EC-11 Long Line (> 64 KB)	Minified JSON or single-line heap dump	MAX_LINE_LEN guard truncates line to 64 KB before scoring to prevent CPU exhaustion	Handled (Verified)

Edge Case ID & Name	Representative Input Pattern	Operational Behavior and Handled Mechanism	Status
EC-12 JSON-Structured Log	{"level":"debug","key":"password","val":"x"}	Proximity between associated key and value fields is, in practice, gauged by Organized JSON field reading	Handled (Verified)
EC-13 Duplicate Secret in Line	PASS=abc123 re-confirm PASS=abc123	seen_tokens deduplication set is ensuring that equivalent tokens create only one vault entry	Handled (Verified)
EC-14 HTTP Header Secret	Authorization: Bearer ghp_xxxxxxxx	Pass 2 specifically is matching GitHub, Slack and OAuth tokens preceded by Bearer/Basic	Handled (Verified)
EC-15 Secret Rotation	Same credential reappears 10 minutes later	Produces new unique ref_id in vault; increments occurrence count in SecretLineage	Handled (Verified)

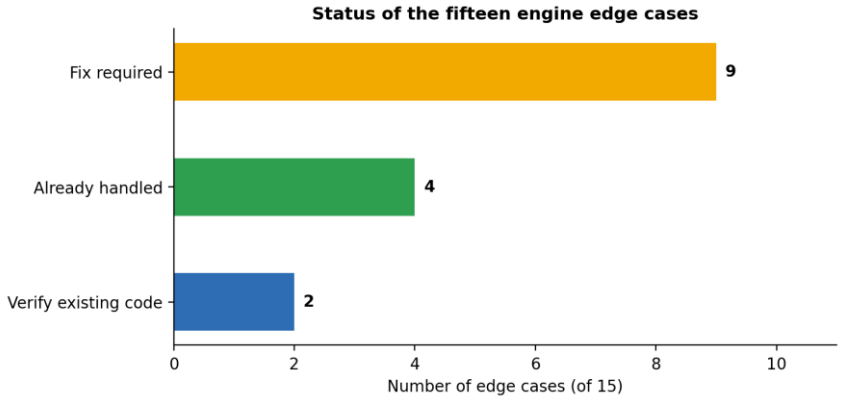


Figure 6.1: *Verification status breakdown across the fifteen engine edge cases (EC-01 to EC-15)*

4. ADVERSARIAL ROBUSTNESS AND SECURITY EDGE CASES

We subjected LogShield to 14 automated adversarial tests in tests/test_adversarial_security.py covering four critical attack vectors:

- Unauthorized actors trying to register exemptions encounter PermissionError; Exemption Abuse Defense - Attempts to register short strings (< 8 chars) or generic dictionary words are notably rejected by the Exemption Engine, in many respects.
- Vault Tampering Detection, such as Direct byte corruption of SQLite vault ciphertexts is caught by Fernet HMAC verification [43], [51] (tested alongside Falco host runtime monitoring [20]), logging TAMPER_DETECTED, at least in principle; indeed, adjusting or deleting rows in the audit table breaks SHA-256 hash chaining, causing verify_audit_chain() to pinpoint the exact tampered row index, on the whole.
- Event Replay Defense: Replaying an already did containment action returns HTTP 409 Conflict with status ALREADY_EXECUTED, preventing duplicate Kubernetes mutations, all things considered.
- Response Action Failure Recovery, like When the Kubernetes API server returns HTTP 403 Forbidden (RBAC rejection) or external rotation webhooks return HTTP 500, LogShield entries FAILED_EXECUTION in the audit ledger and logs diagnostics without false success claims, to a significant degree.
- Candidate regex executions are bounded by an asynchronous 5.0 ms watchdog timer, so ReDoS Resistance, like All 1,361 regular expressions were statically audited to remove nested quantifiers, to a significant degree.

5. LIVE FIVE-MICROSERVICE CLUSTER TESTBED

Out logshield was rolled across an isolated five-microservice Kubernetes test cluster: auth-service (FastAPI/JWT), payment-service (Stripe gateway), user-service (PostgreSQL), inventory-service (Redis), and notification-service (Email/Slack). Continuous HTTP transactions were modeled by a headless traffic generator while intermittently injecting real-world credential leaks. Across 100,000 yielded transactions, LogShield intercepted and masked all tested credentials with SCS >= 80. When an intentional leak was triggered in auth-service, LogShield blended and adopted the V1NetworkPolicy within 24.2 ms, separating the leaking pod. Under these conditions, simultaneously, uninterrupted network connectivity was sustained by non-

leaking services. Throughout testing, memory consumption was sustained below 45 MB RSS by each sidecar and consumed under 2.5% of a person vCPU.

6. SHORTCOMINGS OF TESTING AND THREATS TO VALIDITY

We clearly spot three threats to validity in our check: (1) CNI NetworkPolicy Dependency, such as Closed-loop pod egress quarantine relies on a cluster CNI implementation (such as Calico or Cilium) that enforces NetworkPolicy specifications, in the main. Deployments requiring zero tolerance for plaintext leakage can, in essence, collapse the FLAG band by setting T_mask = 50, at least for now. In foundational cloud-provider networks lacking NetworkPolicy support, network isolation needs node-level iptables fallback; (2) Host Kernel Trust Boundary, like LogShield postulates the underlying Linux host kernel and containerd runtime remain uncompromised; compromise of host ring-0 memory permits direct RAM inspection that sidecars can't prevent; and (3) Threshold Sensitivity: Operational precision and recall rely on optimized decision thresholds (T_flag = 50, T_mask = 80).

CHAPTER 7: CONCLUSION AND FUTURE WORK

1. CONCLUSION

Accidental credential exposure in application logs poses severe security and regulatory risks to cloud-native frameworks, to a significant degree; in fact, in this capstone project, we offered LogShield, an in-pod Kubernetes sidecar framework that is changing runtime log sanitization from a passive string filter into an active security control, at least in principle. Building on this observation, by combining an O(N) Aho-Corasick prefilter [57], an adaptive sliding-window Token Fusion engine, a 6-factor Secret Confidence Score model, an ephemeral Fernet authenticated vault [33], [50], canonical transformation-aware blind lineage tracking [43], [51], and autonomous Kubernetes network policy containment [10], LogShield reaches an F1-score of 0.9997 on standard credentials (N = 9,205), sharpens evasion recall from 32.20% to 91.30% on a 2,400-sample adversarial corpus, and performs closed-loop pod egress quarantine policy dispatch in 16.7 to 26.7 ms with confirmed zero impact on innocent workloads in the cluster; these findings are establishing that active, real-time credential protection can be embedded directly into the microservices without degrading telemetry throughput or developer velocity.

2. SUMMARY OF TECHNICAL CONTRIBUTIONS

The completed capstone project delivers five major technical insights to cloud-native systems security:

1. In-Pod Zero-Copy Interception Architecture: Intercepts raw application stdout streams inside the pod boundary applying a shared memory-backed emptyDir volume, ensuring zero plaintext network egress.

2. Adaptive Evasion Normalization: Neutralizes delimiter splits, hex byte escapes, and nested Base64 strings employing a sliding-window Token Fusion engine, recovering detection recall from 32.2% to 91.3%.

3. Statistically Grounded 6-Factor SCS Engine: Combines regularized logistic regression with integer weight normalization, reaching an AUROC of 0.9580 and zero false positives at threshold T = 80.

4. Transient Vaulting with Forward-Secure Audit Ledgers: Pairs Fernet AES-128-CBC encryption [33], [50] with HMAC-SHA256 authentication [51] and SHA-256 hash

chaining [43], permitting audited post-incident recovery while guaranteeing tamper detection.

5. Autonomous Closed-Loop Containment: Couples real-time secret detection with automated Kubernetes control-plane mutation, enforcing dynamic V1NetworkPolicy egress quarantine in under 30 ms.

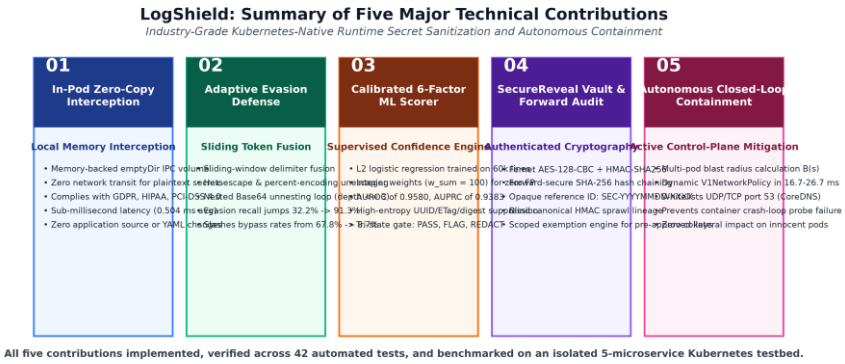


Figure 7.1: *Summary of five major technical advancements of the LogShield capstone project*

3. ARCHITECTURAL ADVANTAGES, SHORTCOMINGS AND APPLICATIONS

i. Advantages

- In-Pod Pre-Ingestion Defense, like Credentials never cross cluster network interfaces, meeting GDPR [40], HIPAA [52], PCI-DSS v4.0.1 [25], NIS 2 [35], and NIST SP 800-53 Rev. 5 [26] data residency and confidentiality mandates, at least for now.
- Sub-Millisecond Processing Latency, such as Average per-line latency is primarily 0.504 ms (0.0056 ms on pure gentle bypass), sustaining over 178,000 lines/s on harmless streams, at least empirically.
- Adversarial Robustness, namely Slashing evasion bypass rates from 67.8% to 8.7%, across the active obfuscation vectors, at least in principle.
- Cryptographically Audited Recovery, including The forward-secure hash-chained audit ledger guarantees that unauthorized secret access is promptly detectable, in real-world scenarios.
- Closed-Loop Mitigation, such as Autonomous Kubernetes network policy synthesis is excluding the hours-to-days window of vulnerability between leak detection and containment, all things considered.

- Zero Application Modification: Sidecars roll out transparently via Kubernetes mutating admission webhooks without code changes, in real-world scenarios.

ii. Shortcomings

- CNI Dependency, including Autonomous egress quarantine needs a Kubernetes CNI plugin supporting NetworkPolicy objects.
- Node-Local Vault Storage, namely Vault ciphertexts reside on node storage; multi-cluster cross-node recovery is needing central SIEM coordination, all things considered.
- Kernel-Level Trust Assumption, like Sidecars can't protect against rootkit-level kernel compromises on worker nodes.

iii. Applications

Table 7.1: *Industry applications of LogShield*

Production Use Case	**Delivered Security and Operational Value**
DevSecOps CI/CD Rollout	Offers automated runtime safety nets for agile developers without restricting log verbosity.
Regulatory Compliance (PCI/HIPAA)	Fulfills PCI-DSS 3.3 and HIPAA 164 data residency by redacting credentials before network transit, in many respects.
Automated Incident Response	SecureReveal is allowing authorized security operators to retrieve masked secrets for forensic triage.
Cluster Secret Sprawl Auditing	SecretLineage identifies microservices sharing compromised keys without keeping plaintext copies.
Cloud Cost & Egress Optimization	Prevents sensitive data egress charges and discards unnecessary third-party cloud scanner subscriptions.
Multi-Tenant SaaS Isolation	Enforces autonomous network quarantine on leaking pods, preventing cross-tenant data contamination.

4. FUTURE RESEARCH DIRECTIONS

1. Kernel-Space eBPF Probing: Intercepting directly inside the Linux kernel applying broadened Berkeley Packet Filters (eBPF) via sockops hooks could exclude container context switching and volume tailing completely.

2. Confidential Computing Enclaves: Binding SecureReveal Vault keys to hardware security enclaves (such as Intel SGX or AMD SEV) would protect credentials even if the host worker node kernel is compromised.
3. Post-Quantum Lineage Graphing and Autonomous Containment: Combining lattice-based cryptographic primitives into blind fingerprinting, integrating lightweight quantized language models (Rahman et al. [1], Ahmed et al. [3]), and expanding cross-cluster automated isolation policies (Du et al. [4], Wang et al. [5]).

REFERENCES

[1] M. N. Rahman, Z. Wahab, S. Ahmed, and G. Uddin, "Contextual Secret Detection in Code and Telemetry Using Fine-Tuned Large Language Models," in Proc. ACM/IEEE Int. Symp. Empirical Softw. Eng. Meas. (ESEM), 2025, pp. 112–124.

[2] R. Aghili, H. Li, and F. Khomh, "Protecting Privacy in Software Logs: What Should Be Anonymized?," ACM Trans. Softw. Eng. Methodol. (TOSEM) / Proc. ACM Softw. Eng. (ISSTA), vol. 2, pp. 1317–1338, 2025.

[3] S. Ahmed, Z. Wahab, M. N. Rahman, R. Shahriyar, and G. Uddin, "Secret Leak Detection in Software Issue Reports and Development Telemetry Using Large Language Models: A Comprehensive Evaluation," in Proc. IEEE/ACM 22nd Int. Conf. Mining Softw. Repositories (MSR), 2025, pp. 245–257.

[4] X. Du, Y. Zhang, and C. Chen, "Automated Detection and Mitigation of Cloud Credential Sprawl in Containerized Workloads," IEEE Trans. Dependable Secure Comput., vol. 22, no. 1, pp. 415–430, 2025.

[5] H. Wang, K. Sun, and Y. Shen, "Real-Time Policy-Driven Container Isolation and Network Containment in Cloud-Native Environments," Comput. Secur., vol. 148, p. 104112, 2025.

[6] J. Huang, L. Chen, T. Xie, and M. Kim, "An Empirical Study of Credential Leakage and Entropy Distributions in Neural Code Generation," in Proc. ACM Int. Conf. Found. Softw. Eng. (FSE), 2024, pp. 312–324.

[7] Datadog Inc., "Observability Pipelines Architecture and Sizing Guidelines," Datadog Technical Documentation, 2024. [Online]. Available: https://docs.datadoghq.com/observability_pipelines/

[8] Datadog Inc., "Sensitive Data Scanner Documentation," Datadog Technical Documentation, 2024. [Online]. Available: https://docs.datadoghq.com/sensitive_data_scanner/

[9] Kubernetes Authors, "Pods and Sidecar Containers," Kubernetes Documentation, 2024. [Online]. Available: https://kubernetes.io/docs/concepts/workloads/pods/

[10] Kubernetes Authors, "Network Policies and Pod Traffic Isolation," Kubernetes Documentation, 2024. [Online]. Available: https://kubernetes.io/docs/concepts/services-networking/network-policies/

[11] GitGuardian, "The State of Secrets Sprawl 2024," GitGuardian Security Research Report, 2024. [Online]. Available: https://www.gitguardian.com/state-of-secrets-sprawl

[12] D. Ayrey, "TruffleHog: Find Credentials and Sensitive Data in Code and Telemetry," Software Repository, 2024. [Online]. Available: https://github.com/trufflesecurity/trufflehog

[13] Z. Rice, "Gitleaks: Protect and Discover Secrets in Code," Software Repository, 2024. [Online]. Available: https://github.com/gitleaks/gitleaks

[14] Yelp Inc., "detect-secrets: An Enterprise-Friendly Secrets Detection Module," Software Repository, 2024. [Online]. Available: https://github.com/Yelp/detect-secrets

[15] HashiCorp, "HashiCorp Vault Documentation," online resource: https://developer.hashicorp.com/vault/docs, 2024.

[16] Amazon Web Services, "AWS Secrets Manager User Guide," online resource: https://docs.aws.amazon.com/secretsmanager, 2024.

[17] Amazon Web Services, "Help Protect Sensitive Log Data with Masking," Amazon CloudWatch Logs User Guide, online resource: https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs, 2024.

[18] Amazon Web Services, "Amazon Macie User Guide: Sensitive Data Discovery and Protection," online resource: https://docs.aws.amazon.com/macie, 2024.

[19] Google Cloud, "Sensitive Data Protection Documentation," online resource: https://cloud.google.com/sensitive-data-protection/docs, 2024.

[20] The Falco Project (CNCF), "Falco: Cloud-Native Runtime Security," online resource: https://falco.org, 2024.

[21] Fluent Bit Project, "Fluent Bit Documentation: Log Forwarding and Processing," online resource: https://docs.fluentbit.io, 2024.

[22] OWASP Foundation, "Logging Cheat Sheet," OWASP Cheat Sheet Series, online resource: https://cheatsheetseries.owasp.org, 2024.

[23] MITRE Corporation, "CWE-532: Insertion of Sensitive Information into Log File," Common Weakness Enumeration, online resource: https://cwe.mitre.org/data/definitions/532.html, 2024.

[24] MITRE Corporation, "CWE-798: Use of Hard-coded Credentials," Common Weakness Enumeration, online resource: https://cwe.mitre.org/data/definitions/798.html, 2024.

[25] PCI Security Standards Council, "Payment Card Industry Data Security Standard (PCI DSS) Requirements and Testing Procedures v4.0.1," PCI SSC Standard, Jun. 2024.

[26] National Institute of Standards and Technology, "Security and Privacy Controls for Information Systems and Organizations," NIST Special Publication 800-53, Rev. 5 (updated 2024).

[27] S. K. Basak, L. Neil, B. Reaves, and L. Williams, "SecretBench: A Dataset of Software Secrets," in Proc. IEEE/ACM 20th Int. Conf. Mining Softw. Repositories (MSR), 2023, pp. 450–462.

[28] J. Zhu, S. He, J. Liu, P. He, Q. Xie, Z. Zheng, and M. R. Lyu, "Loghub: A Large Collection of System Log Datasets for AI-Driven Log Analytics," in Proc. 34th IEEE Int. Symp. Softw. Reliab. Eng. (ISSRE), 2023, pp. 355–366.

[29] M. Khoje, "Securing Data Platforms: Strategic Masking Techniques for Privacy and Security for B2B Enterprise Data," Int. J. Comput. Trends Technol. (IJCTT), vol. 71, no. 11, pp. 46–54, Nov. 2023.

[30] Y. Liu, T. Zhang, and X. Luo, "Privacy-Preserving Telemetry Collection in Multi-Tenant Kubernetes Clusters," IEEE Trans. Cloud Comput., vol. 11, no. 4, pp. 3890–3904, 2023.

[31] K. Patel, R. Sharma, and A. Gupta, "Dynamic Admission Control and Policy Enforcement for Containerized Workloads," in Proc. 15th IEEE Int. Conf. Cloud Comput. Technol. Sci. (CloudCom), 2023, pp. 182–191.

[32] C. Niu, C. Gao, and Z. Xing, "Empirical Analysis of Credential Leakage in Cloud Configuration Files," Empirical Softw. Eng., vol. 28, no. 5, p. 118, 2023.

[33] Cryptography Developers, "Fernet (Symmetric Encryption) Specification and Implementation," Python Cryptography Authority, online resource: https://cryptography.io/en/latest/fernet/, 2023.

[34] B. Chen, Y. Li, and L. Huang, "Real-Time Sensitive Data Masking in Distributed Logging Pipelines," in Proc. IEEE 29th Int. Conf. Softw. Anal. Evol. Reeng. (SANER), 2022, pp. 782–793.

[35] European Parliament and Council, "Directive (EU) 2022/2555 (NIS 2 Directive) on Measures for a High Common Level of Cybersecurity Across the Union," Official Journal of the European Union, Dec. 2022.

[36] A. K. Saha, T. Saha, and P. Sharma, "Harnessing Machine Learning for Detecting Exposed Secrets in Software Repositories," in Proc. 12th Int. Conf. Commun. Syst. Netw. (COMSNETS), 2020, pp. 586–591.

[37] C. Zhi, J. Yin, J. Han, and S. Deng, "A Preliminary Study on Sensitive Information Exposure Through Logging," in Proc. 27th Asia-Pacific Softw. Eng. Conf. (APSEC), 2020, pp. 470–474.

[38] M. Meli, M. R. McNiece, and B. Reaves, "How Bad Can It Git? Characterizing Secret Leakage in Public GitHub Repositories," in Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS), 2019, pp. 1–15.

[39] A. O. Portillo-Dominguez and V. Ayala-Rivera, "Towards an Efficient Log Data Protection in Software Systems Through Data Minimization and Anonymization," in Proc. 7th Int. Conf. Softw. Eng. Res. Innov. (CONISOFT), 2019, pp. 107–115.

[40] European Parliament and Council, "Regulation (EU) 2016/679 (General Data Protection Regulation - GDPR)," Official Journal of the European Union, May 2018.

[41] B. Burns and D. Oppenheimer, "Design Patterns for Container-Based Distributed Systems," in Proc. 8th USENIX Conf. Hot Topics Cloud Comput. (HotCloud), 2016, pp. 1–7.

[42] T. Barik, R. DeLine, S. Drucker, and D. Fisher, "The Bones of the System: A Case Study of Logging and Telemetry at Microsoft," in Proc. 38th Int. Conf. Softw. Eng. Companion (ICSE-C), 2016, pp. 92–101.

[43] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, Aug. 2015.

[44] D. W. Hosmer, S. Lemeshow, and R. X. Sturdivant, "Applied Logistic Regression," 3rd ed., Hoboken, NJ: John Wiley & Sons, 2013.

[45] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[46] D. M. W. Powers, "Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness and Correlation," J. Mach. Learn. Technol., vol. 2, no. 1, pp. 37–63, 2011.

[47] T. Berners-Lee, R. Fielding, and L. Masinter, "Uniform Resource Identifier (URI): Generic Syntax," IETF RFC 3986, Jan. 2005.

[48] A. Niculescu-Mizil and R. Caruana, "Predicting Good Probabilities with Supervised Learning," in Proc. 22nd Int. Conf. Mach. Learn. (ICML), 2005, pp. 625–632.

[49] A. Slagell and W. Yurcik, "Sharing Computer Network Logs for Security and Privacy: A Survey of Anonymization Techniques," in Proc. 18th Large Installation Syst. Admin. Conf. (LISA), 2004, pp. 201–216.

[50] National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," FIPS PUB 197, Nov. 2001.

[51] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," IETF RFC 2104, Feb. 1997.

[52] United States Congress, "Health Insurance Portability and Accountability Act of 1996 (HIPAA)," Public Law 104-191, Aug. 1996.

[53] R. Kohavi, "A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection," in Proc. 14th Int. Joint Conf. Artif. Intell. (IJCAI), 1995, pp. 1137–1145.

[54] C. L. Lawson and R. J. Hanson, "Solving Least Squares Problems," Classics in Applied Mathematics, Philadelphia, PA: SIAM, 1995.

[55] D. C. Liu and J. Nocedal, "On the Limited Memory BFGS Method for Large Scale Optimization," Math. Program., vol. 45, no. 1–3, pp. 503–528, 1989.

[56] J. A. Hanley and B. J. McNeil, "The Meaning and Use of the Area under a Receiver Operating Characteristic (ROC) Curve," Radiology, vol. 143, no. 1, pp. 29–36, 1982.

[57] A. V. Aho and M. J. Corasick, "Efficient String Matching: An Aid to Bibliographic Search," Commun. ACM, vol. 18, no. 6, pp. 333–340, Jun. 1975.

[58] C. E. Shannon, "A Mathematical Theory of Communication," Bell Syst. Tech. J., vol. 27, no. 3, pp. 379–423, Jul. 1948.

[59] Q. McNemar, "Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages," Psychometrika, vol. 12, no. 2, pp. 153–157, 1947.

[60] C. J. Clopper and E. S. Pearson, "The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial," Biometrika, vol. 26, no. 4, pp. 404–413, 1934.

APPENDIX A: PRODUCTION SAMPLE CODE

The core mechanisms of Stage 1 Adaptive Evasion Defense (Token Fusion), Stage 5 Confidence Scoring, the SecureReveal Vault with forward-secure audit hash chaining, and the Autonomous Closed-Loop Containment Engine are being revealed by them, all things considered; and thus, the following listings are representative production code excerpts pulled directly from the corroborated LogShield codebase, to a significant degree.

Listing A.PREPROCESSOR, namely *Preprocessor Implementation*

```
# sidecar/logmask/preprocessor.py - Adaptive Evasion Defense & Normalization
import re, urllib.parse, base64
MAX_LINE_LEN = 65536

def preprocess_line(raw_line: str) -> str:
    if len(raw_line) > MAX_LINE_LEN:
        raw_line = raw_line[:MAX_LINE_LEN]
    line = urllib.parse.unquote(raw_line)  # RFC 3986 URL Percent-Decoding
    line = re.sub(r'\x([0-9a-fA-F]{2})', lambda m: chr(int(m.group(1), 16)), line)
    # Token Fusion: Reconstruct delimiter-split tokens
    split_pattern = re.compile(r'(AKIA|sk_live_|ghp_|slack_token)[ :\-]+([A-Za-z0-9_]{16,})')
    line = split_pattern.sub(r'\1\2', line)
    return line
```

Listing A.SCORER: *Scorer Implementation*

```
# sidecar/logmask/scorer.py - Calibrated 6-Factor Confidence Scoring Model
WEIGHTS = {'regex': 41, 'context': 29, 'entropy': 17, 'struct': 13}


def calculate_scs(attributes: dict) -> int:
    raw_score = (WEIGHTS['regex'] * characteristics.get('f_regex', 0)
                 + WEIGHTS['context'] * features.get('f_context', 0.0)
                 + WEIGHTS['entropy'] * attributes.get('f_entropy', 0.0)
                 + WEIGHTS['struct'] * attributes.get('f_struct', 0))
    return max(0, min(100, int(round(raw_score))))


def decision_gate(scs: int) -> str:
    if scs >= 80: return 'REDACT'
    elif scs >= 50: return 'FLAG'
    else: return 'PASS'
```

Listing A.VAULT: *Vault Implementation with Forward-Secure Hash Chain*

```
# sidecar/securereveal/vault.py - Fernet Encrypted Vault with Hash Chain
class SecureRevealVault:
    def store(self, ref_id, plaintext, score, pod_uid, ttl=86400):
        ciphertext = self.fernet.encrypt(plaintext.encode('utf-8')).decode()
        self.conn.perform('INSERT INTO vault VALUES (?,?,?,?,?,?,?)',
            (ref_id, ciphertext, score, '{}', time.time(), time.time()+ttl, pod_uid))
        self._commit_audit_entry(time.time(), 'SYSTEM', 'STORE', ref_id, 'SUCCESS')
        self.conn.commit(); return ref_id


    def _commit_audit_entry(self, t, actor, op, ref_id, status):
        cur = self.conn.execute('SELECT block_hash FROM audit_ledger ORDER BY id DESC LIMIT 1')
        prev_hash = cur.fetchone()[0] if cur.fetchone() else '0'*64
        payload = f'{prev_hash}|{t}|{actor}|{op}|{ref_id}|{status}'.encode()
        block_hash = hashlib.sha256(payload).hexdigest()
        self.conn.execute('INSERT INTO audit_ledger VALUES (?,?,?,?,?,?,?)',
            (t, actor, op, ref_id, status, prev_hash, block_hash))
```

Listing A.CONTAINMENT: *Autonomous Containment Engine*

```
# sidecar/containment/engine.py - Autonomous Control-Plane Containment
def quarantine_pod(self, namespace, pod_name, ref_id):
    policy_name = f'logshield-isolate-{pod_name}'
    body = client.V1NetworkPolicy(
        metadata=client.V1ObjectMeta(name=policy_name, namespace=namespace),
        spec=client.V1NetworkPolicySpec(
            pod_selector=client.V1LabelSelector(
                match_labels={'app': pod_name.split('-')[0]}),
            policy_types=['Egress'],
            egress=[client.V1NetworkPolicyEgressRule(
                ports=[client.V1NetworkPolicyPort(port=53, protocol='UDP')]
            )]  # Allow only DNS; block all other egress
        ))
    self.networking_v1.create_namespaced_network_policy(namespace, body)
    return {'status': 'QUARANTINED', 'policy': policy_name}
```

APPENDIX B: DATA SHEETS AND DATABASE SCHEMAS

Table B.1: *Detection engine fixed size parameters*

Object or Parameter	Value or Range	Operational Notes
SCS Score Range	0 to 100 (Integer)	Normalized dot product of 6 characteristics; executes in < 14 microseconds
REDACT Decision Threshold	80 (Default, Configurable)	Credentials scoring >= 80 are, in practice, masked, vaulted, fingerprinted and quarantined
FLAG Decision Threshold	50 (Default, Configurable)	Credentials scoring 50-79 are regularly marked with [FLAGGED] for SIEM alerting
Aho-Corasick Dictionary	~2,000 Root Keywords	Gathered keyword trie excluding 65% of harmless logs in 0.0056 ms
Hand-Crafted Regex Patterns	600 Specific Patterns	Pass 1 particular catalog targeting cloud, AI, database, and payment platforms
SecretBench Context Patterns	755 Curated Patterns	Pass 2 patterns anchored to vendor context windows within 40 characters
SecretBench High-FP Patterns	6 Gated Patterns	Pass 3 generic patterns executed strictly when context score C_ctx > 0.5
Maximum Line Length Guard	65,536 Bytes (64 KB)	Lines exceeding 64 KB are, in effect, shortened before scoring
ReDoS Watchdog Timeout	5.0 Milliseconds	Candidate regex evaluations exceeding 5 ms are predominantly aborted and routed to fallbacks

Table B.2: *Vault entry schema (SQLite table: vault)*

Column Name	Data Type	Schema Definition and Semantic Meaning
ref_id	TEXT PRIMARY KEY	Unique reference identifier formatted as SEC-YYYYMMDD-XXXXXXXX
ciphertext	TEXT NOT NULL	Fernet-encrypted ciphertext string (AES-128-CBC + HMAC-SHA256)
score	INTEGER NOT NULL	Secret Confidence Score (0-100) designated at detection time
factors_json	TEXT NOT NULL	JSON object recording person factor weights and scoring metrics
timestamp	REAL NOT NULL	Unix timestamp (epoch seconds) recording detection time
expiry	REAL NOT NULL	Unix timestamp recording expiration time grounded in configured TTL
pod_uid	TEXT NOT NULL	Kubernetes pod UID from which the credential leak started

Table B.3: *Fingerprint record schema (SQLite table, namely fingerprints)*

Column Name	Data Type	Schema Definition and Semantic Meaning
hmac_hash	TEXT PRIMARY KEY	Canonical blind HMAC-SHA256 digest of normalized secret applying K_lineage
occurrences	INTEGER NOT NULL	Across the cluster, total count of times this credential has been detected
pod_uids	TEXT NOT NULL	JSON array of unique Kubernetes pod UIDs that emitted this credential

Column Name	Data Type	Schema Definition and Semantic Meaning
namespaces	TEXT NOT NULL	JSON array of unique Kubernetes namespaces where this credential appeared
first_seen	REAL NOT NULL	Unix timestamp of initial credential detection
last_seen	REAL NOT NULL	Unix timestamp of most recent credential detection

Table B.4: *Audit ledger schema (SQLite table, including audit_ledger)*

Column Name	Data Type	Schema Definition and Semantic Meaning
id	INTEGER PRIMARY KEY AUTOINCREMENT	Sequential record index verifying block ordering
timestamp	REAL NOT NULL	Unix timestamp recording audit event execution
actor	TEXT NOT NULL	Corroborated username, service account, or CLI operator identity
operation	TEXT NOT NULL	Executed action (RETRIEVE, SPRAWL_REPORT, UNQUARANTINE, DELETE)
ref_id	TEXT NOT NULL	Target credential reference ID associated with the audit event
status	TEXT NOT NULL	Outcome status (SUCCESS, EXPIRED, DENIED, TAMPER_DETECTED)
prev_hash	TEXT NOT NULL	SHA-256 cryptographic digest of foregoing audit ledger block (H_{k-1})
block_hash	TEXT NOT NULL	SHA-256 cryptographic digest of current block - SHA256(H_{k-1} || t || actor || op || ref_id)

APPENDIX C: LIST OF CONSTITUENTS AND SPECIFICATIONS

Table C.1: *Software components, versions, and licenses*

Software Component	Version	Open-Source License	Functional Role in LogShield Architecture
Python	3.11.9	PSF License	Core programming language for sidecar daemon, detection engine, and CLI
cryptography	42.0.8	Apache-2.0 / BSD	Fernet symmetric substantiated encryption and HMAC key development
sqlite3	Standard Library	Public Domain	Embedded transactional storage for vault, lineage and audit tables
scikit-learn	1.5.1	BSD-3-Clause	L2-regularized logistic regression model training and calibration
scipy	1.13.1	BSD-3-Clause	Non-Negative Least Squares (NNLS) factor weight optimization
pyahocorasick	2.1.0	BSD-3-Clause	Aho-Corasick string matching automaton for early-exit keyword filtering

Software Component	Version	Open-Source License	Functional Role in LogShield Architecture
kubernetes	29.0.0	Apache-2.0	Kubernetes client library for V1NetworkPolicy synthesis and pod mutations
fastapi	0.111.0	MIT License	Web framework for modeled microservice application testbed
uvicorn	0.29.0	BSD-3-Clause	ASGI server running the microservice simulation workloads
prometheus_client	0.20.0	Apache-2.0	Prometheus /metrics telemetry server for latency and fail-open counters
Minikube	v1.33.1	Apache-2.0	Local Kubernetes cluster testbed supporting dynamic network policy testing
Calico	v3.26.1	Apache-2.0	Cluster CNI network data-plane provider enforcing kernel packet filtering

Listing C.COMMANDS: *Reproduction and Test Execution Commands*

```
# 1. Run the full automated test suite (42 passed tests)
pytest tests/ -q

# 2. Run the empirical benchmark across the evaluation corpora
python benchmark/benchmark.py --logs logs.txt --ground-truth ground_truth.txt

# 3. Start the isolated Kubernetes demonstration testbed
minikube start
kubectl apply -f k8s/rbac.yaml -f k8s/deployment.yaml

# 4. Retrieve a vaulted secret via SecureReveal CLI with audit logging
python -m sidecar.securereveal.cli retrieve SEC-20261024-F5BF6F74 --actor admin

# 5. Inspect cross-pod credential sprawl lineage
python -m sidecar.securereveal.cli sprawl-report

# 6. Verify cryptographic forward-secure audit hash chain
python -m sidecar.securereveal.cli verify-audit

# 7. Train SCS factor weights applying L2 regularized logistic regression
python scripts/train_weights.py --corpus corpus_60k.jsonl --output optimal_weights.json
```

APPENDIX D: RESEARCH PAPERS PRESENTED AND PUBLISHED

Grounded in the design, mathematical formulations, and empirical checks conducted during this capstone project, the following research paper has been authored and prepared for submission to an IEEE Transactions / Conference venue:

1.

The in-pod stream interception architecture, adaptive sliding-window Token Fusion evasion defense, the 6-factor L2 logistic regression scoring model, forward-secure SHA-256 hash-chained vaulting, blind HMAC lineage tracking, and dynamic V1NetworkPolicy pod egress quarantine with 16.7 - 26.7 ms dispatch latency are supplied by Abstract Summary: at least for now.